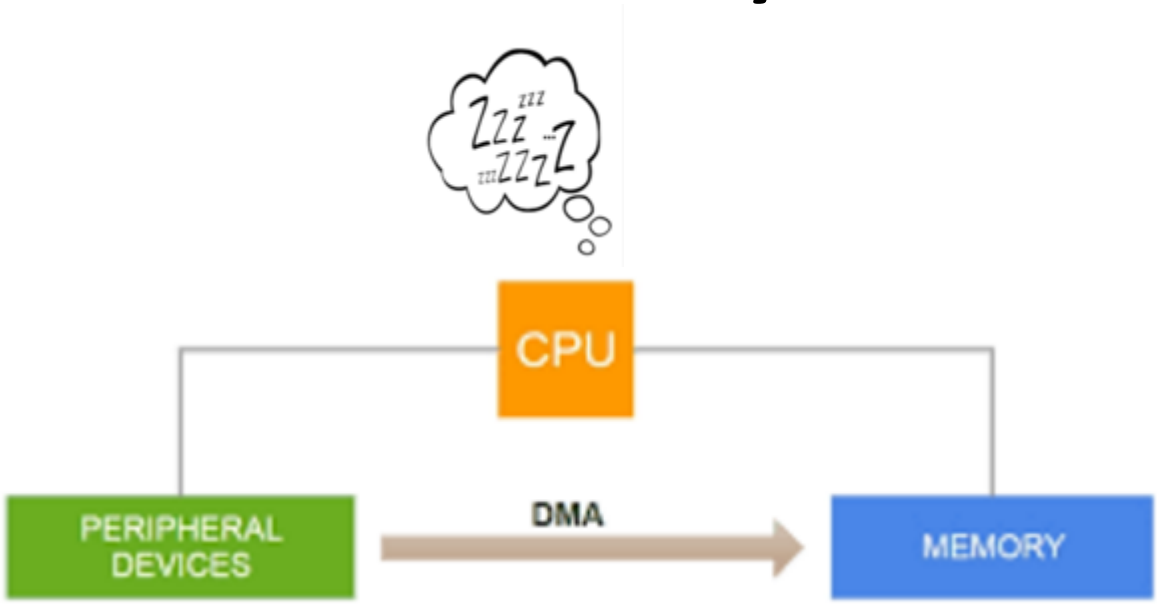
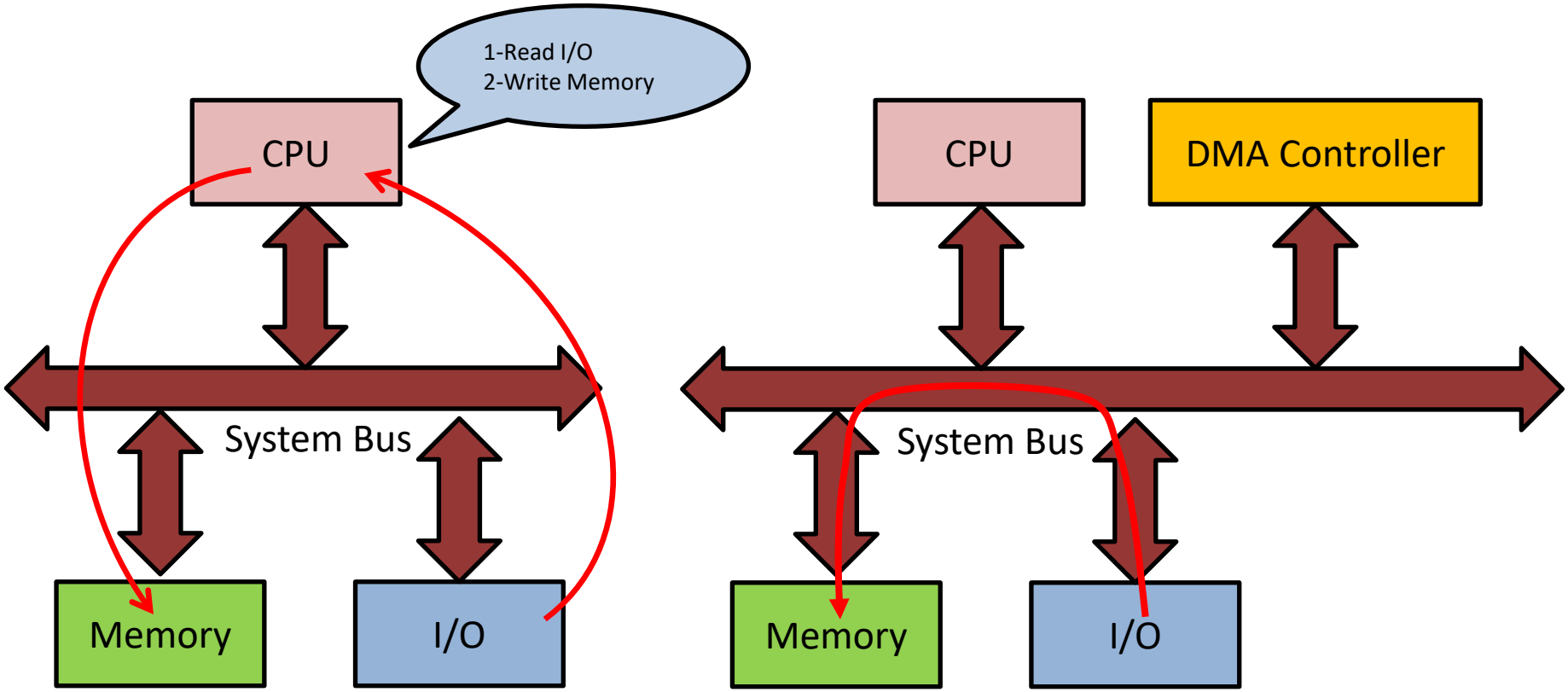
DMA
Direct Memory Access



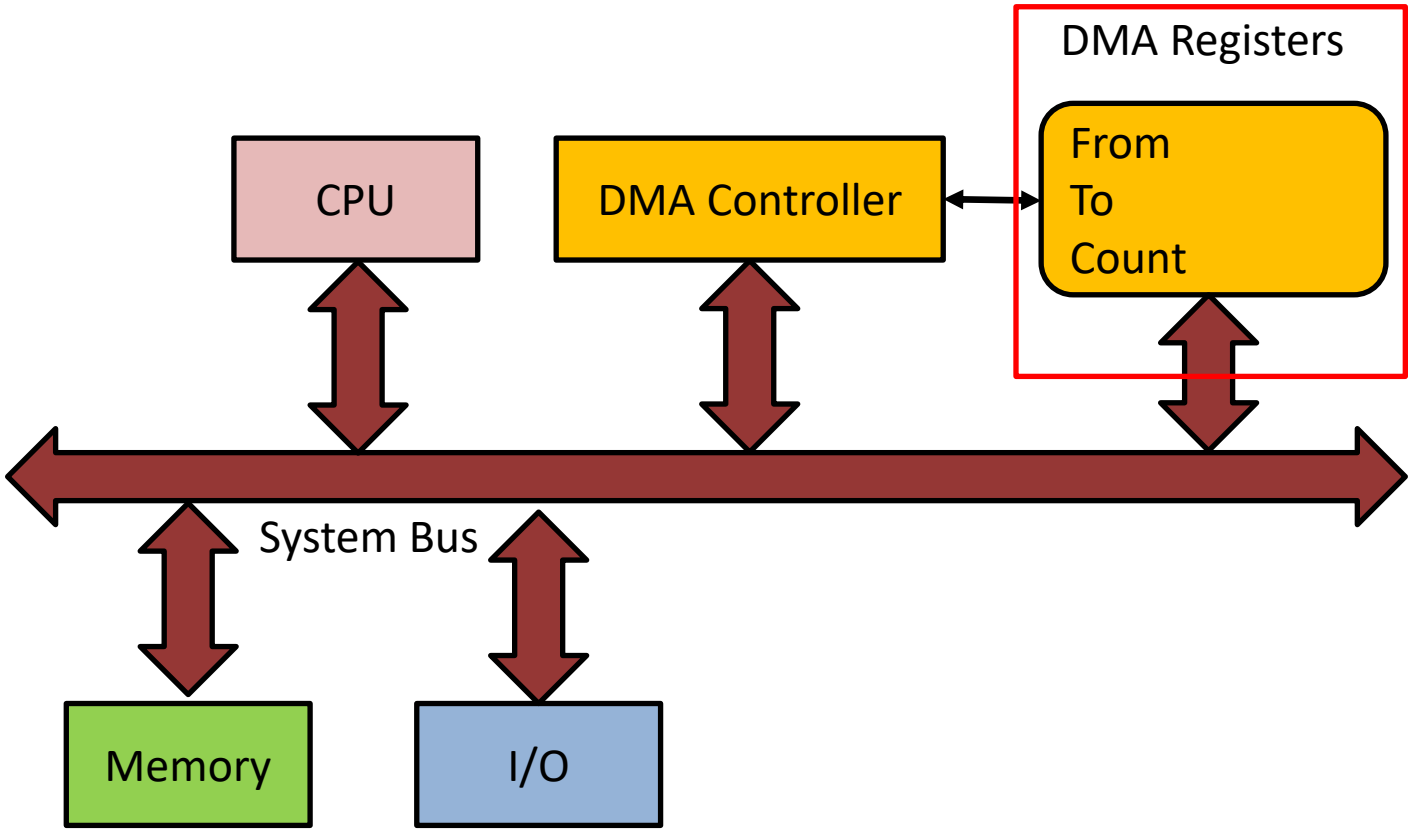
Que es ?



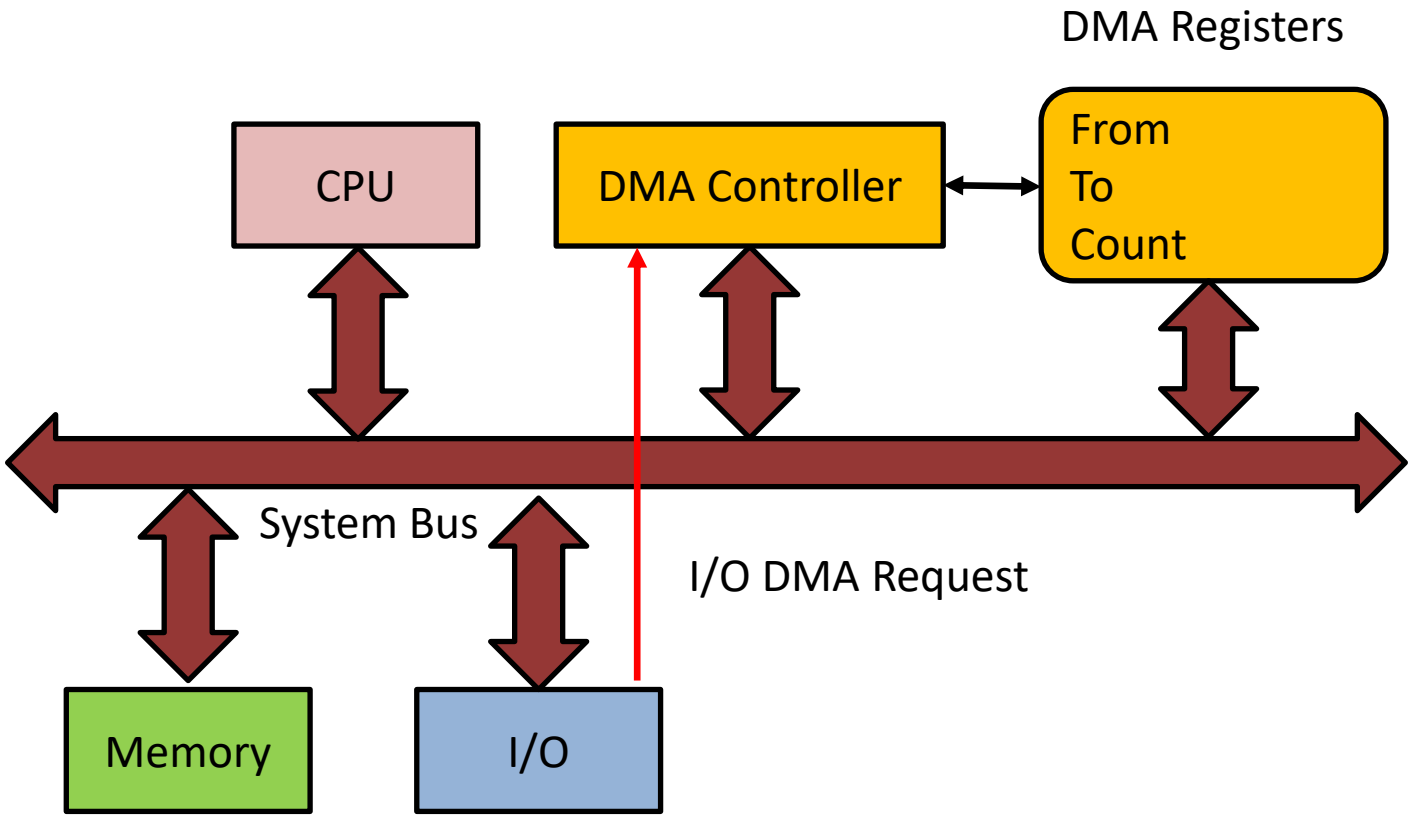
(a) Programmed I/O Transfer

(b) DMA Transfer

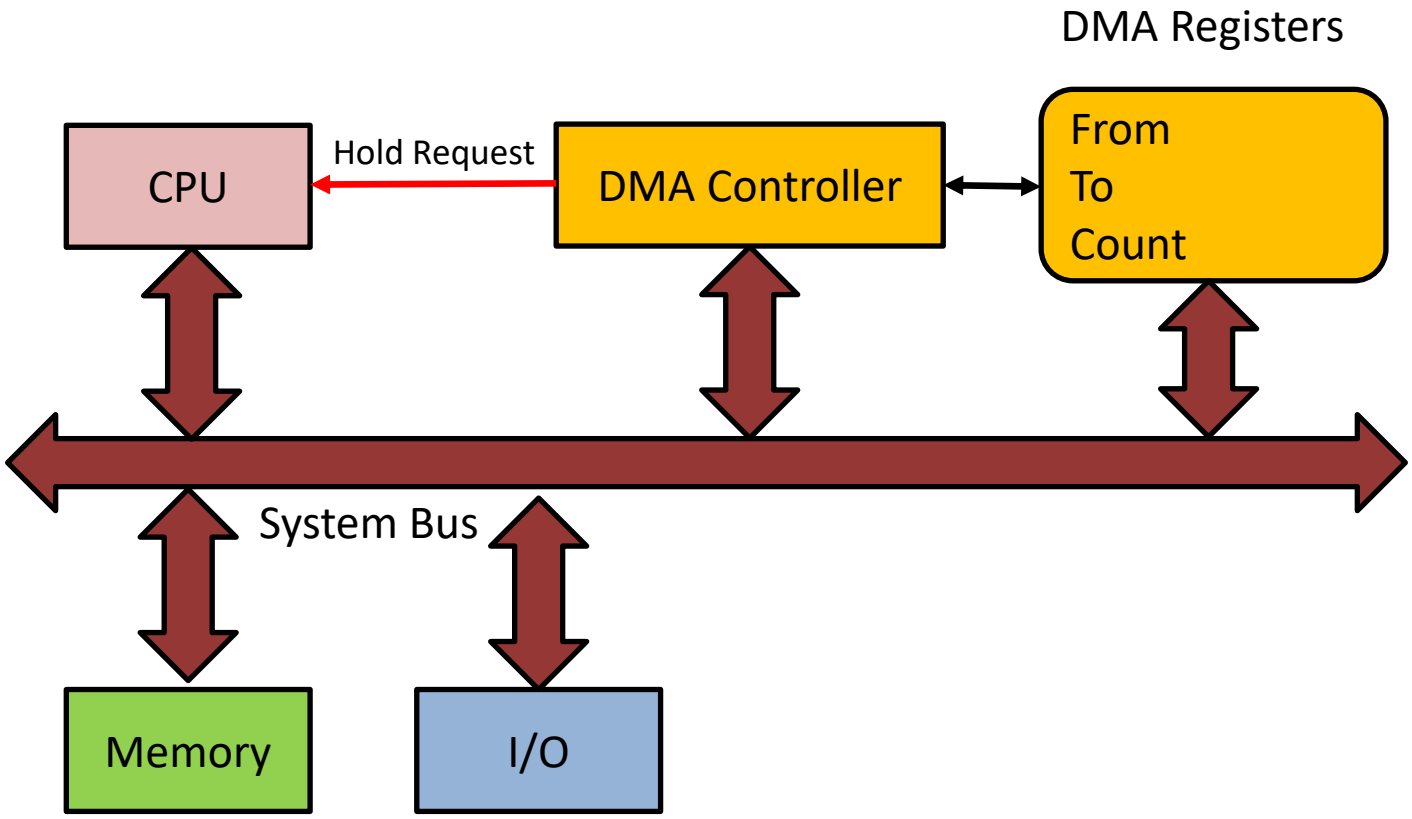
1-Configuramos el controlador



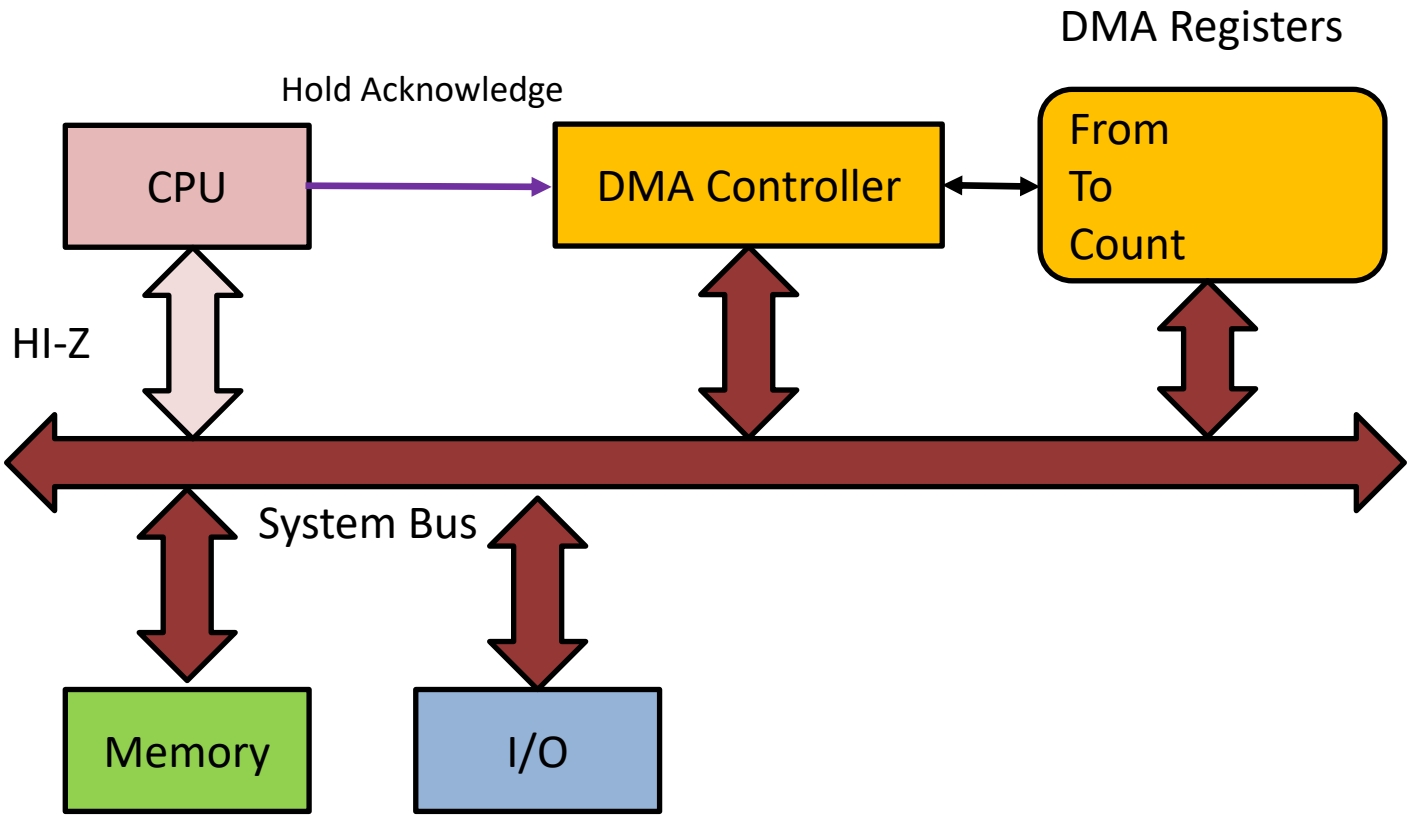
2-El periférico solicita una transferencia



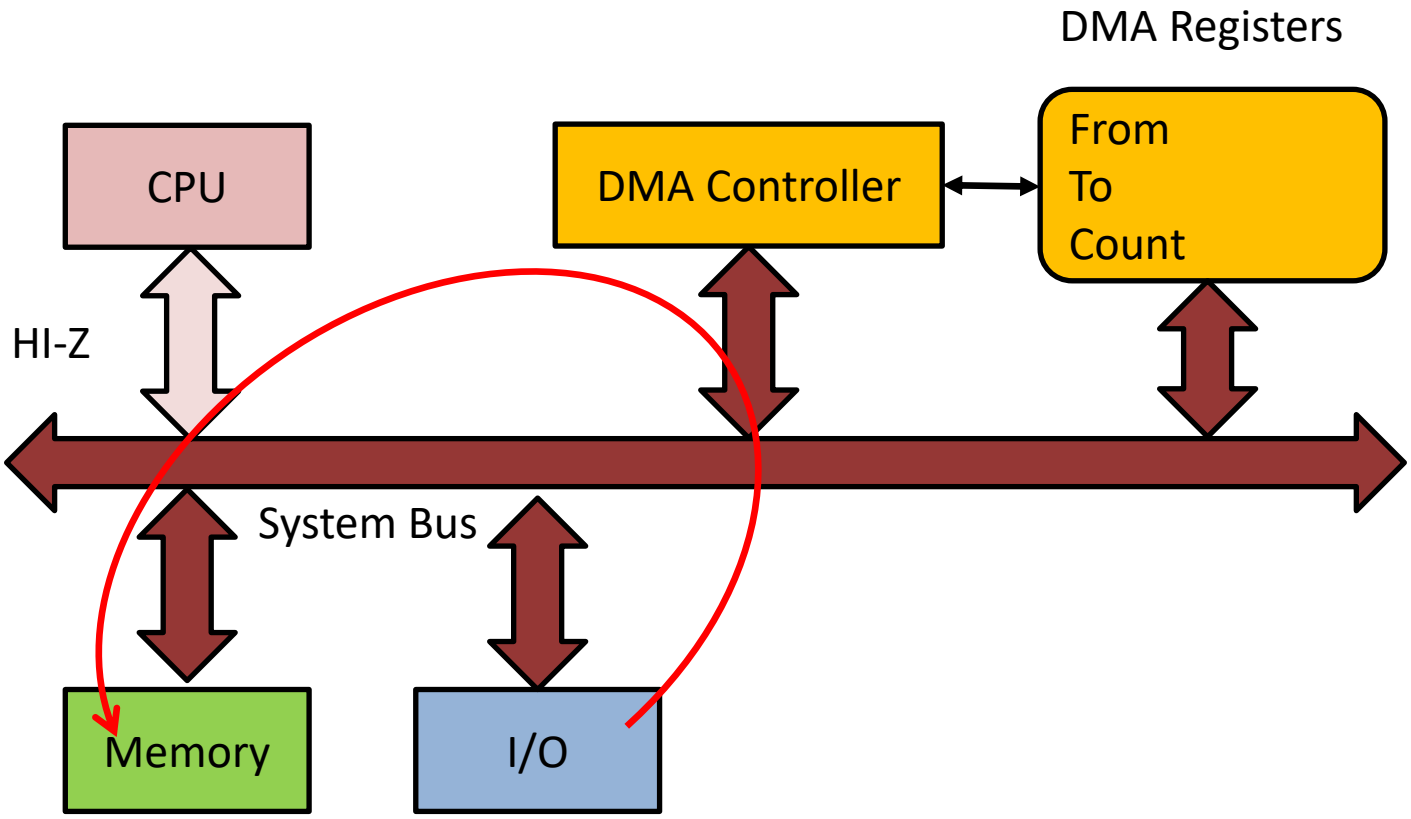
3-El DMA Controlador de DMA solicita el bus a la CPU



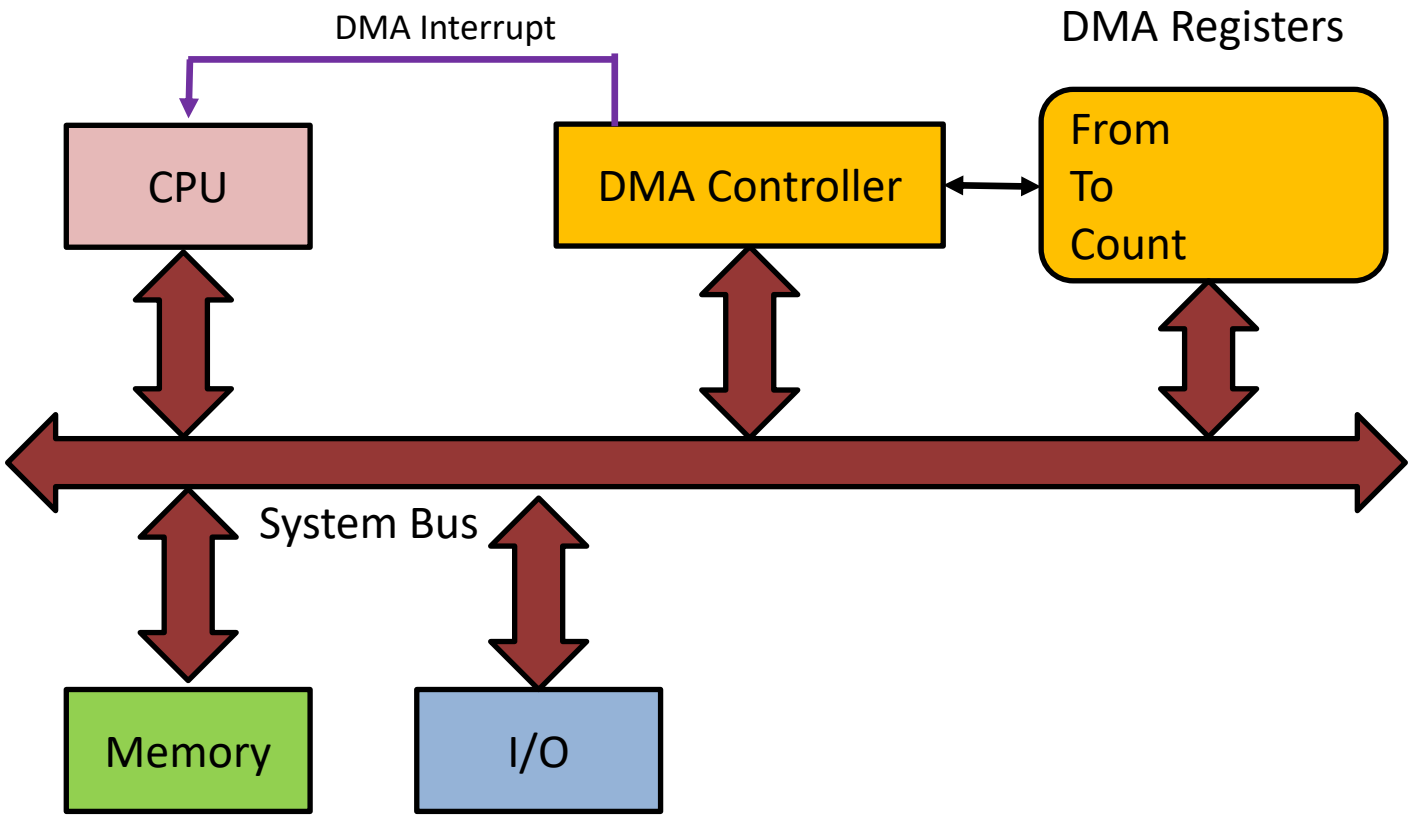
4-La CPU accede al pedido



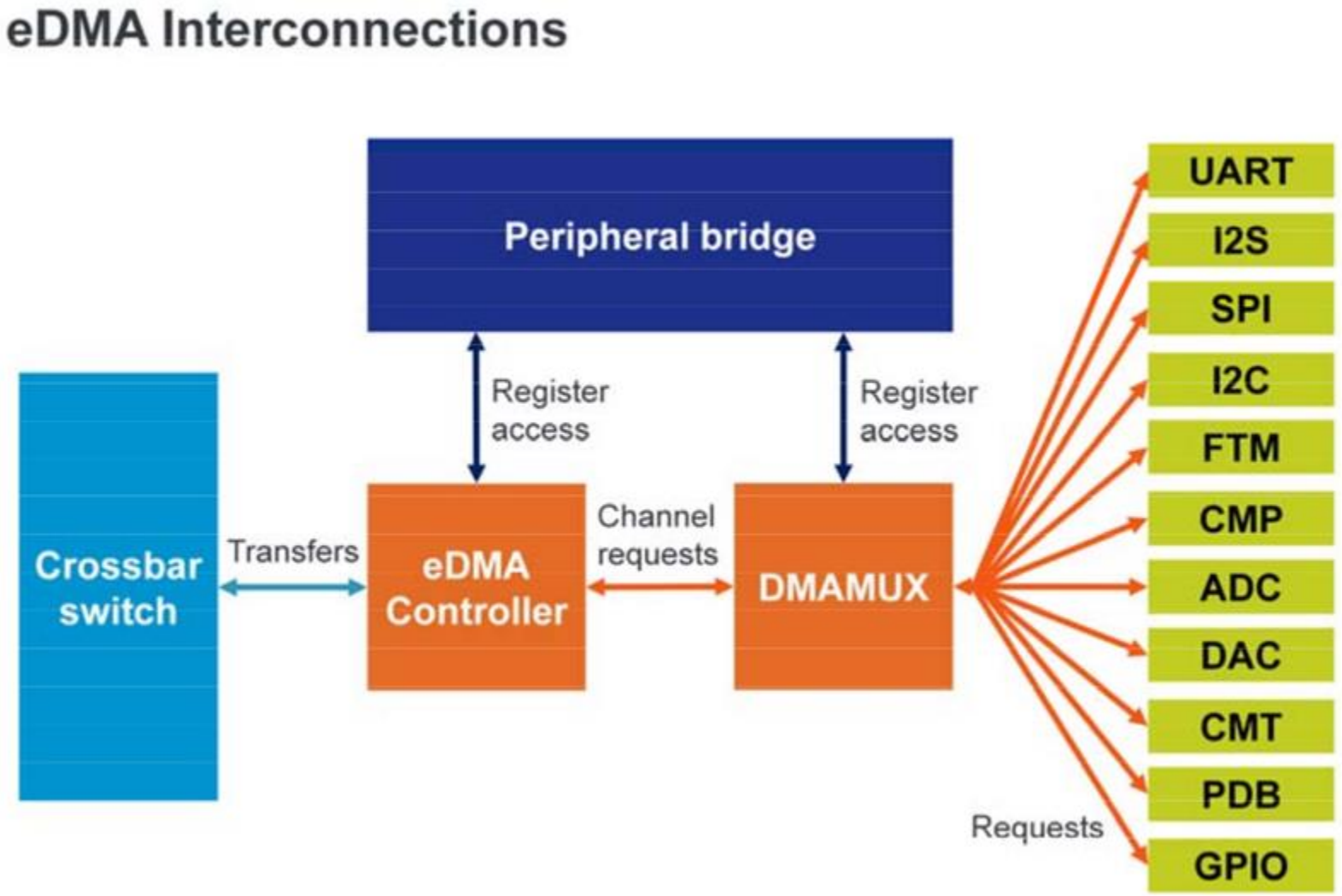
5-Se inicia la transferencia



6-La transferencia finaliza



K64 DMA Block Diagram



DMA & DMAMUX Clock Source

Module	Bus interface clock	Internal clocks	I/O interface clocks
System modules			
DMA	System clock	—	—
DMA Mux	Bus clock	—	—

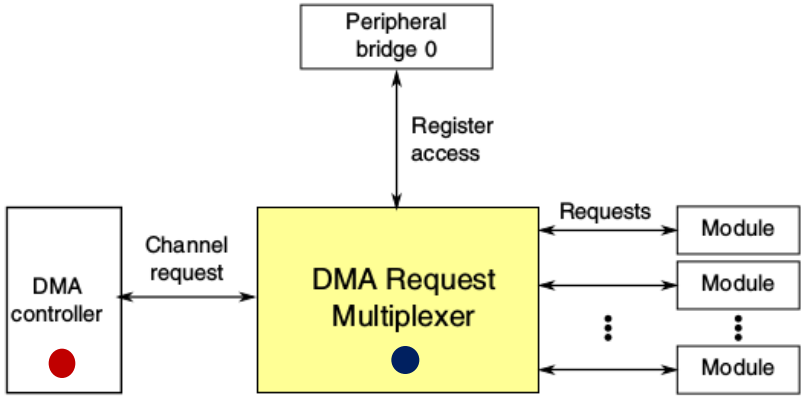


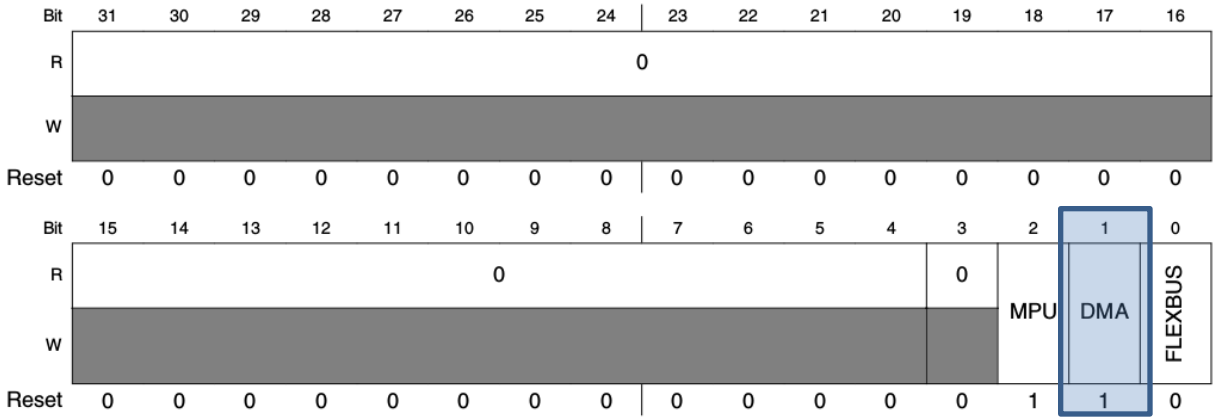
Figure 3-14. DMA request multiplexer configuration

Topic	Related module	Reference
Full description	DMA request multiplexer	DMA Mux
System memory map		System memory map
Clocking		Clock distribution
Power management		Power management
Channel request	DMA controller	DMA Controller
Requests		DMA request sources

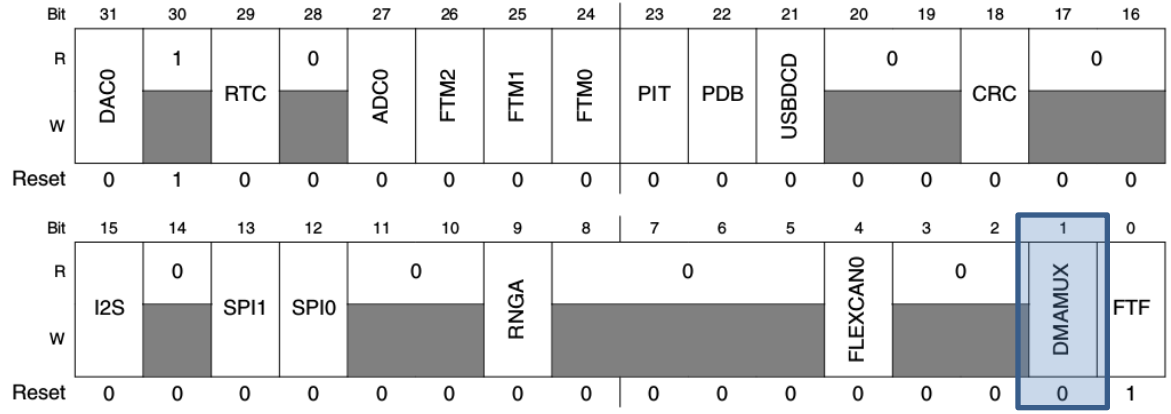


Pag. 93 K64 Sub-Family Reference Manual, Rev. 2, January 2014

DMA & DMAMUX Gating



SIM_SCGC7 field descriptions



SIM_SCGC6 field descriptions

DMA MUX

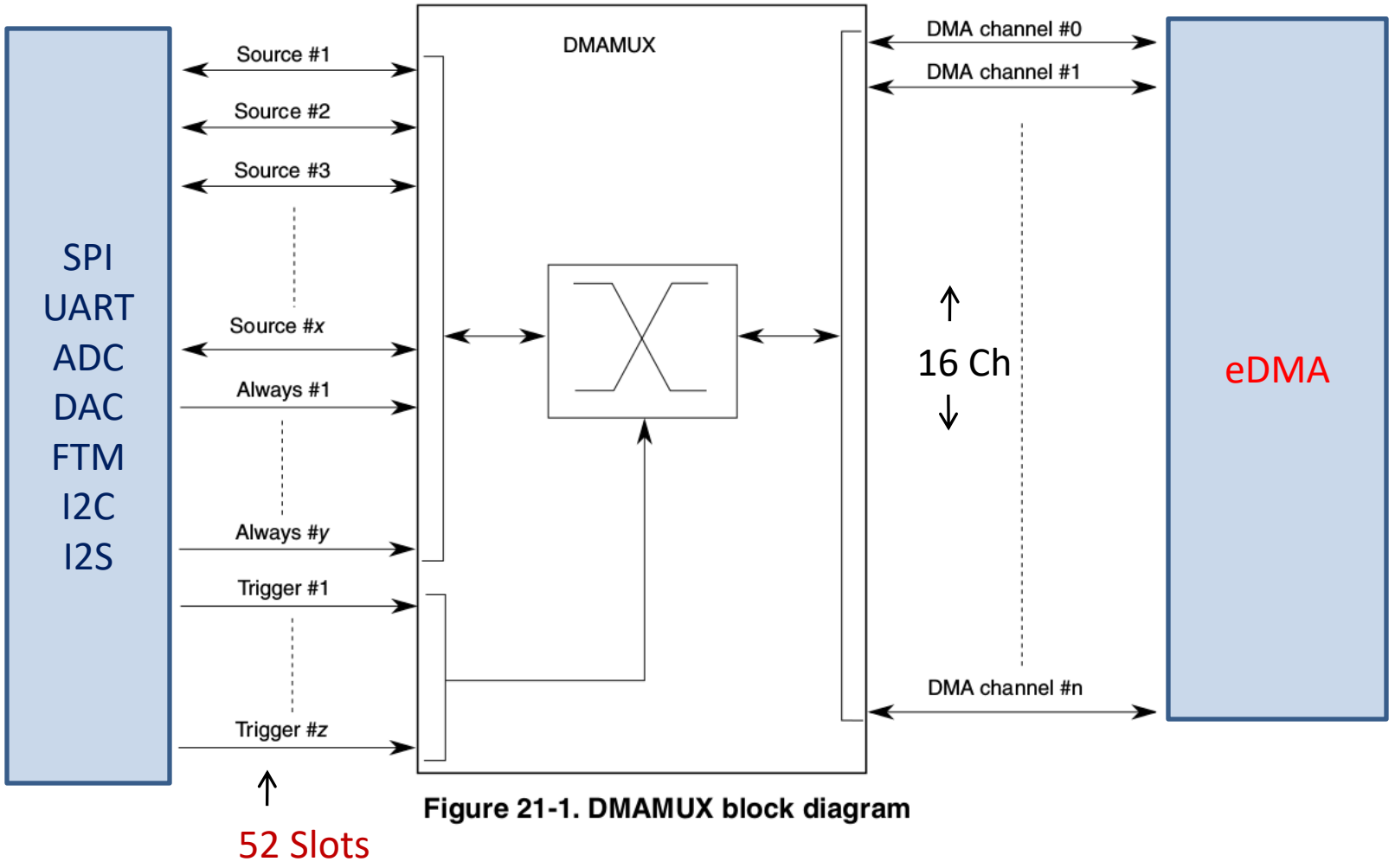


Figure 21-1. DMAMUX block diagram

DMA MUX 1/3

Table 3-27. DMA request sources - MUX 0

Source number	Source module	Source description
0	—	Channel disabled[1]
1	Reserved	Not used
2	UART0	Receive
3	UART0	Transmit
4	UART1	Receive
5	UART1	Transmit
6	UART2	Receive
7	UART2	Transmit
8	UART3	Receive
9	UART3	Transmit
10	UART4	Transmit or Receive
11	UART5	Transmit or Receive
12	I^2S0	Receive
13	I^2S0	Transmit
14	SPI0	Receive
15	SPI0	Transmit
16	SPI1	Transmit or Receive
17	SPI2	Transmit or Receive
18	I^2C0	—
19	I^2C1 or I^2C2	—

DMA MUX 2/3

Source number	Source module	Source description
20	FTM0	Channel 0
21	FTM0	Channel 1
22	FTM0	Channel 2
23	FTM0	Channel 3
24	FTM0	Channel 4
25	FTM0	Channel 5
26	FTM0	Channel 6
27	FTM0	Channel 7
28	FTM1	Channel 0
29	FTM1	Channel 1
30	FTM2	Channel 0
31	FTM2	Channel 1
32	FTM3	Channel 0
33	FTM3	Channel 1
34	FTM3	Channel 2
35	FTM3	Channel 3
36	FTM3	Channel 4
37	FTM3	Channel 5
38	FTM3	Channel 6
39	FTM3	Channel 7

DMA MUX 3/3

Source number	Source module	Source description
40	ADC0	—
41	ADC1	—
42	CMP0	—
43	CMP1	—
44	CMP2	—
45	DAC0	—
46	DAC1	—
47	CMT	—
48	PDB	—
49	Port control module	Port A
50	Port control module	Port B
51	Port control module	Port C
52	Port control module	Port D
53	Port control module	Port E
54	IEEE 1588 Timers	Timer 0
55	IEEE 1588 Timers	Timer 1
56	IEEE 1588 Timers	Timer 2
57	IEEE 1588 Timers	Timer 3
58	DMA MUX	Always enabled
59	DMA MUX	Always enabled
60	DMA MUX	Always enabled
61	DMA MUX	Always enabled
62	DMA MUX	Always enabled
63	DMA MUX	Always enabled

DMA MUX Modes

Disabled: Reconfiguration /Configuration
Normal: DMA Source → DMA Ch
Periodic: Source is Sampled periodically (PIT)

Bit	7	6	5	4	3	2	1	0
Read Write	ENBL	TRIG	SOURCE					
Reset	0	0	0	0	0	0	0	0

DMAMUX_CHCFG*n* field descriptions

ENBL: (= 0) DMA channel is disabled. Should be used to disable or reconfigure a DMA
(= 1) DMA channel is enabled

TRIG: Enables the periodic trigger capability for the triggered DMA channel.
(= 0) Triggering is disabled. If triggering is disabled and ENBL is set, the DMA Channel will simply route the specified source to the DMA channel. (Normal mode)
(= 1) Triggering is enabled. If triggering is enabled and ENBL is set, the DMAMUX is in Periodic Trigger mode.

SOURCE(5-0): DMA Channel Source (Slot) Specifies which DMA source, if any, is routed to a particular DMA channel. See your device's chip configuration details for information about the peripherals and their slot numbers.

DMA channels with periodic triggering capability.

- **Only 4 Channels**

Table 3-61. PIT channel assignments for periodic DMA triggering

DMA Channel Number	PIT Channel
DMA Channel 0	PIT Channel 0
DMA Channel 1	PIT Channel 1
DMA Channel 2	PIT Channel 2
DMA Channel 3	PIT Channel 3

- Gating : This trigger works by gating the request from the peripheral to the DMA until a trigger event has been seen.
- After the DMA request has been serviced, the peripheral will negate its request.

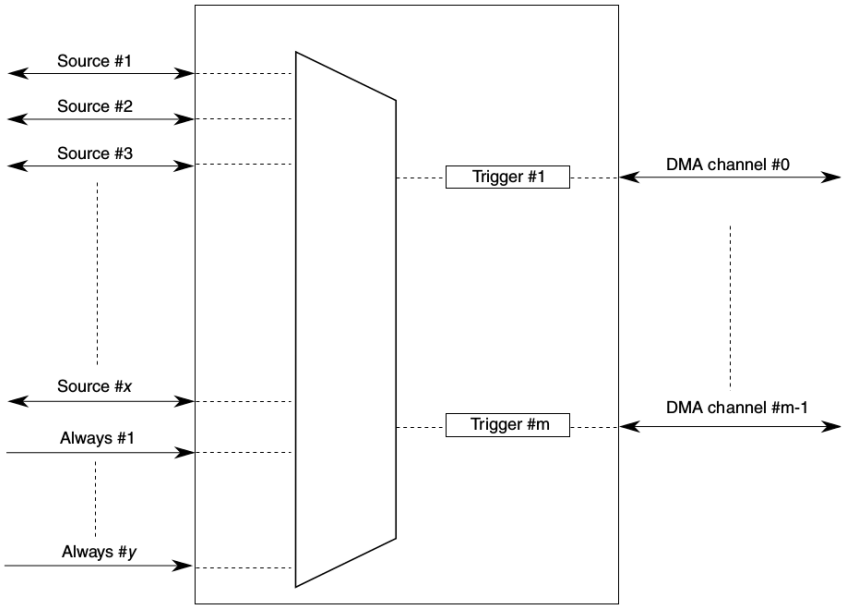


Figure 21-19. DMAMUX triggered channels

DMA channels with periodic triggering capability.

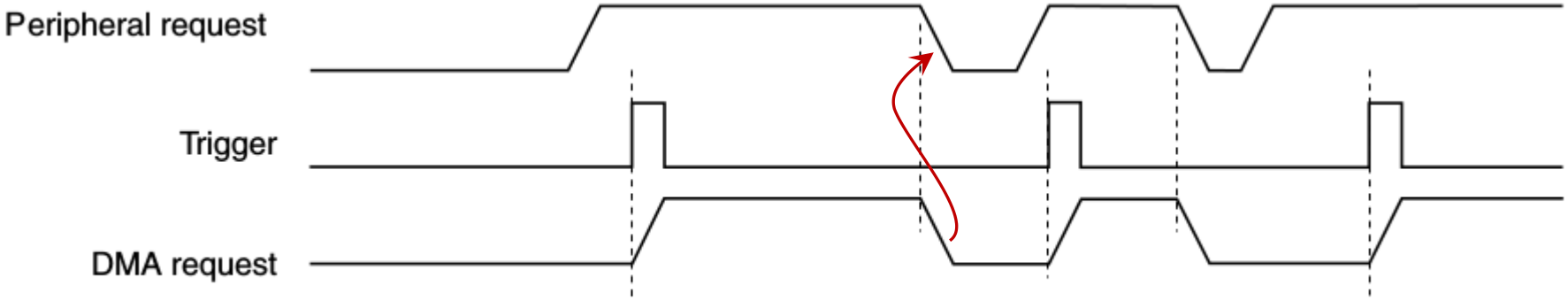


Figure 21-20. DMAMUX channel triggering: normal operation

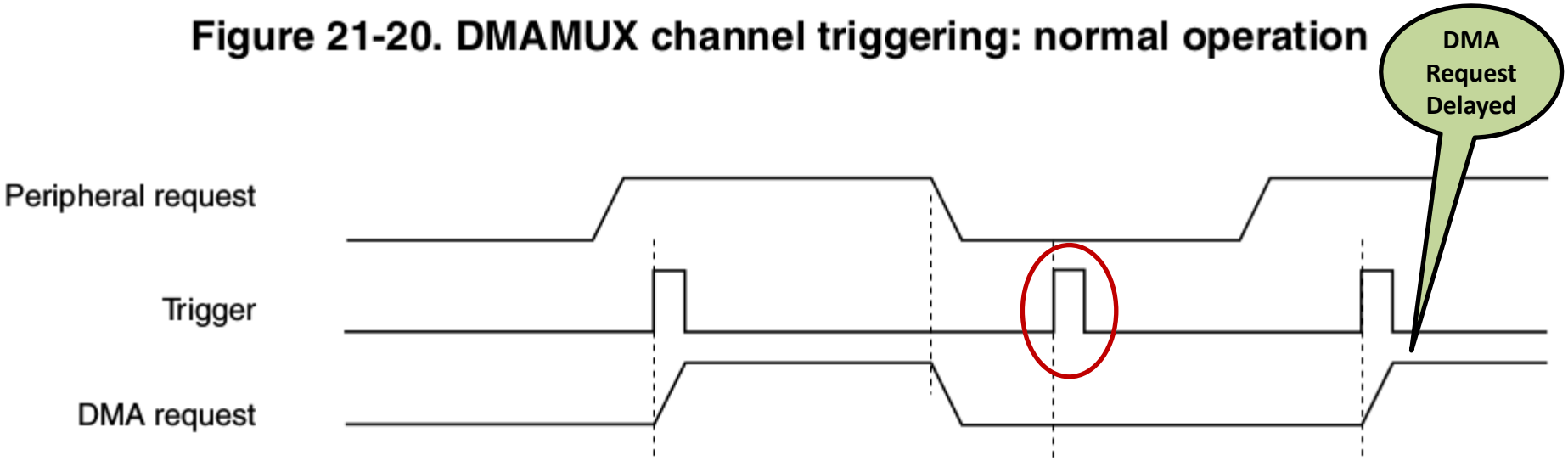


Figure 21-21. DMAMUX channel triggering: ignored trigger

PIT: **P**rogramable **I**nterval **T**imer

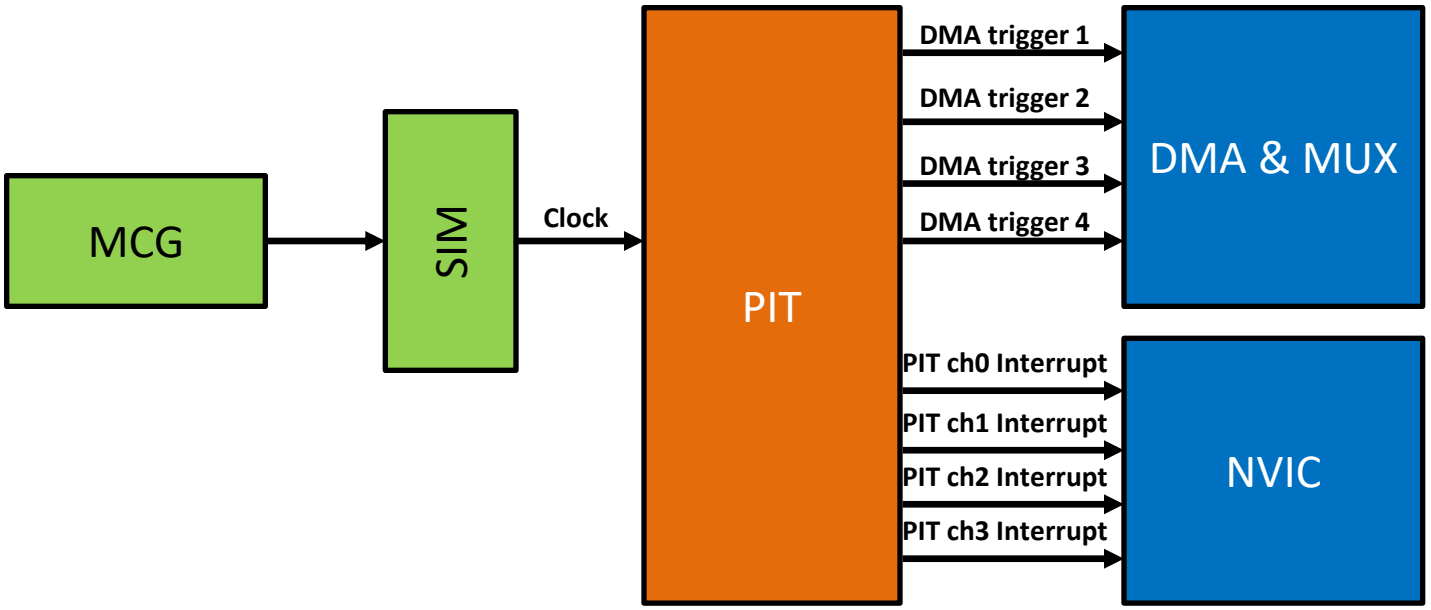
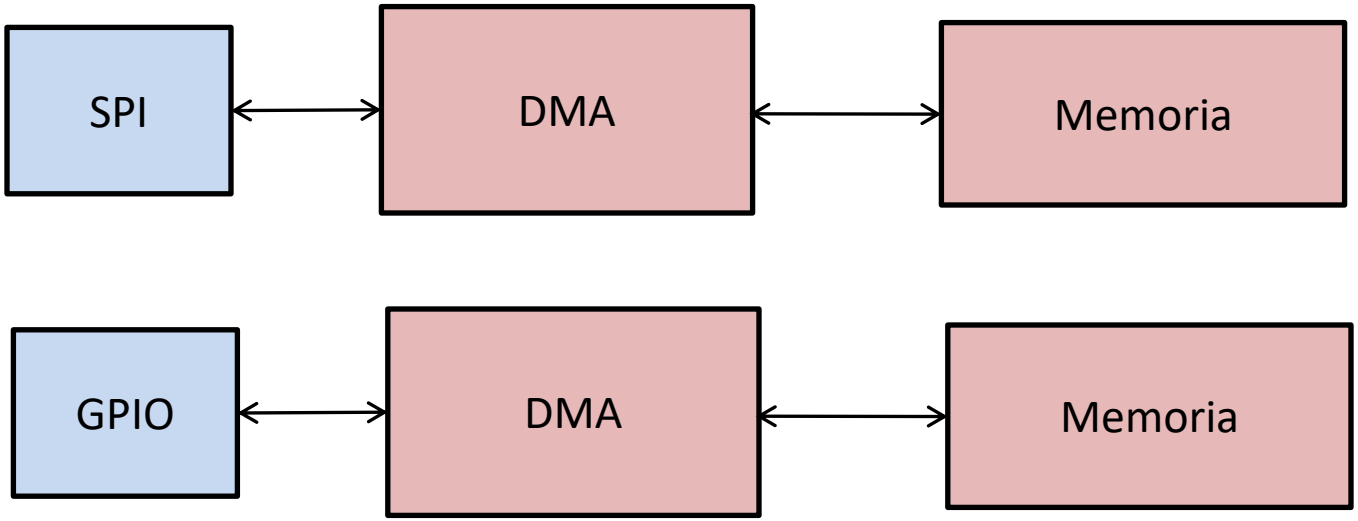


Table 3-61. PIT channel assignments for periodic DMA triggering

DMA Channel Number	PIT Channel
DMA Channel 0	PIT Channel 0
DMA Channel 1	PIT Channel 1
DMA Channel 2	PIT Channel 2
DMA Channel 3	PIT Channel 3

DMA channels with periodic triggering capability.

- Ej: SPI Tx Y Rx Cada 5 useg
- GPIO Generar formas complejas desde una tabla en memoria.



/* Enable the eDMA channel 0 and set the SPI as the DMA request source with periodic trigger */

```
DMAMUX_CHCFG0 |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(16) | DMAMUX_CHCFG_TRIG_MASK;
```

Always-enabled DMA sources

- 10 Channels
- Fast (no Throttling).
- Memory to Memory
- GPIO Fast
- Software Activation

Gated Vs Always on

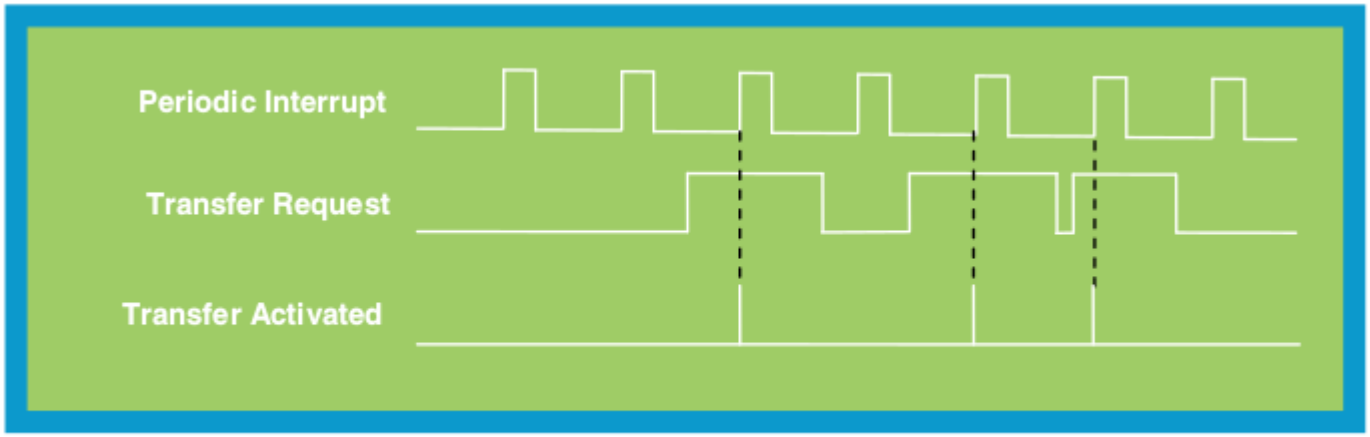


Figure 4. PIT gated transfer activation (Throttled)

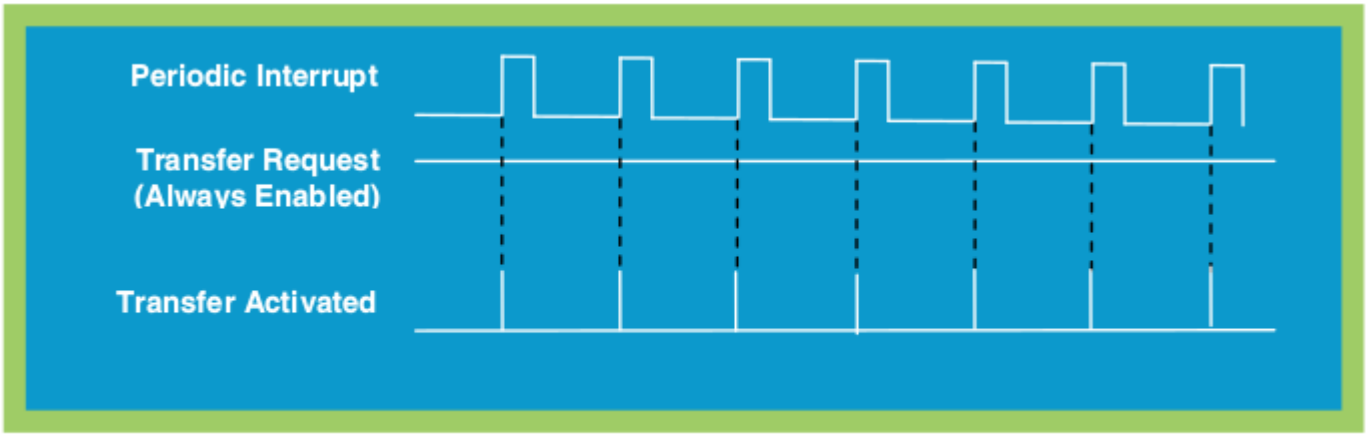


Figure 5. PIT only transfer activation

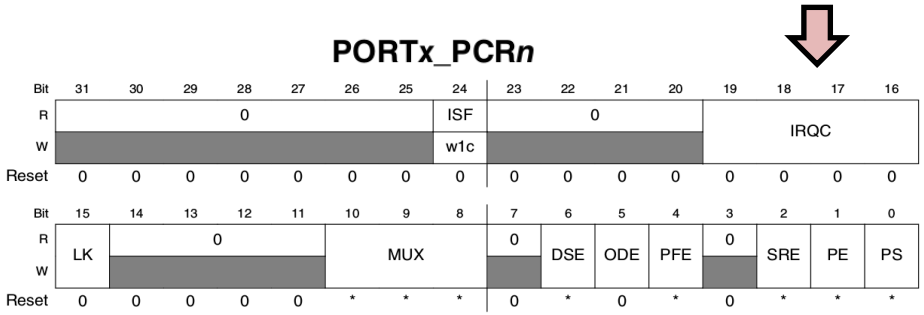
Sample Code

```
/* Configure the SW2 DMA request source on falling edge. */

PORTC_PCR6 |= PORT_PCR_MUX(0x01) | PORT_PCR_IRQC(0x02) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;

/* Enable the eDMA channel 0 and set the PORTC as the DMA request source. */

DMAMUX_CHCFG0 |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(51);
```



IRQC

0000	Interrupt/DMA request disabled.
0001	DMA request on rising edge.
0010	DMA request on falling edge.
0011	DMA request on either edge.
1000	Interrupt when logic 0.
1001	Interrupt on rising-edge.
1010	Interrupt on falling-edge.
1011	Interrupt on either edge.
1100	Interrupt when logic 1.
Others	Reserved.

eDMA - DMA Controller

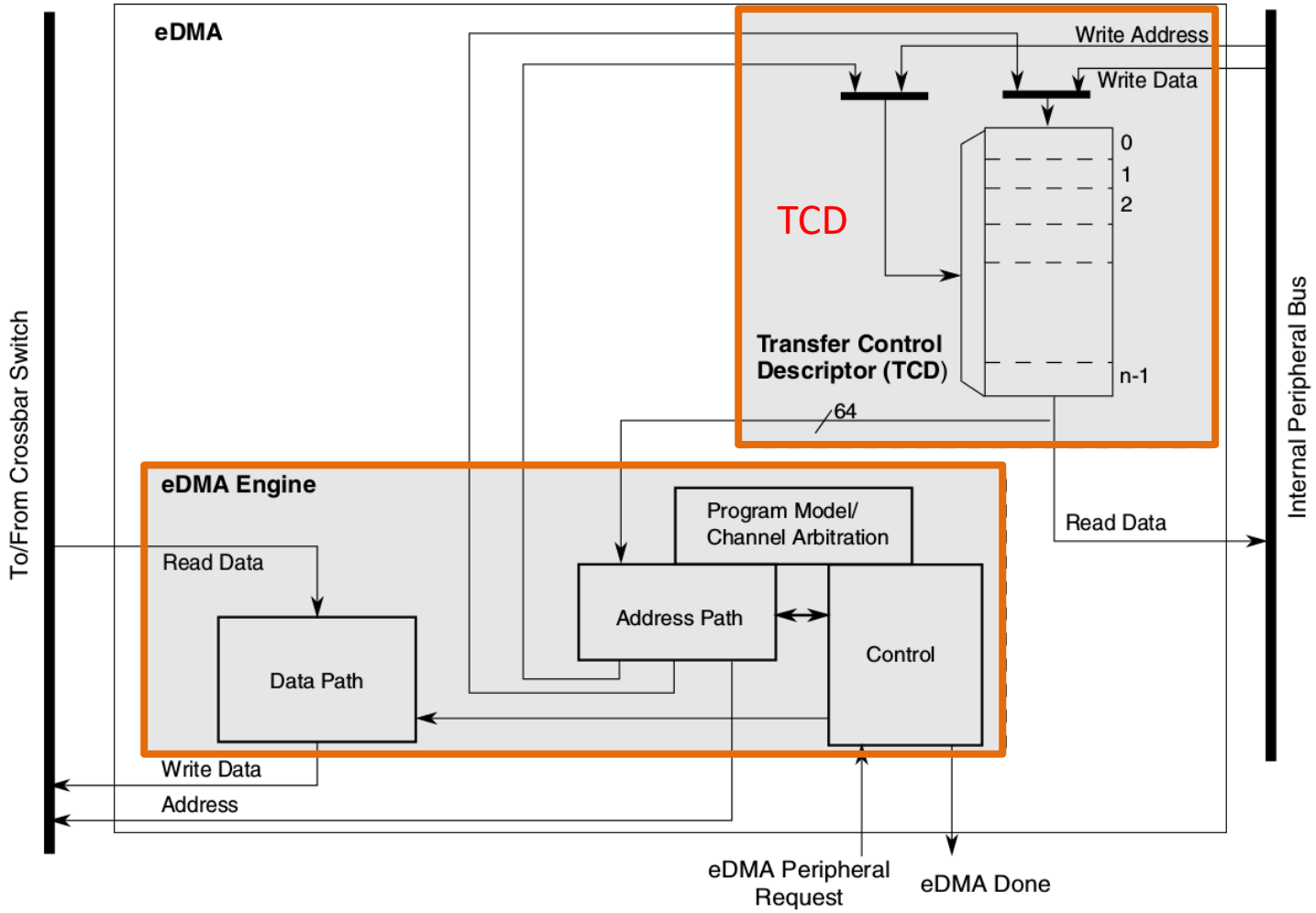


Figure 22-1. eDMA block diagram

eDMA - DMA Controller

La Unidad **eDMA** realiza las siguientes funciones:

- Calculo de la dirección origen y destino
- La transferencia de datos

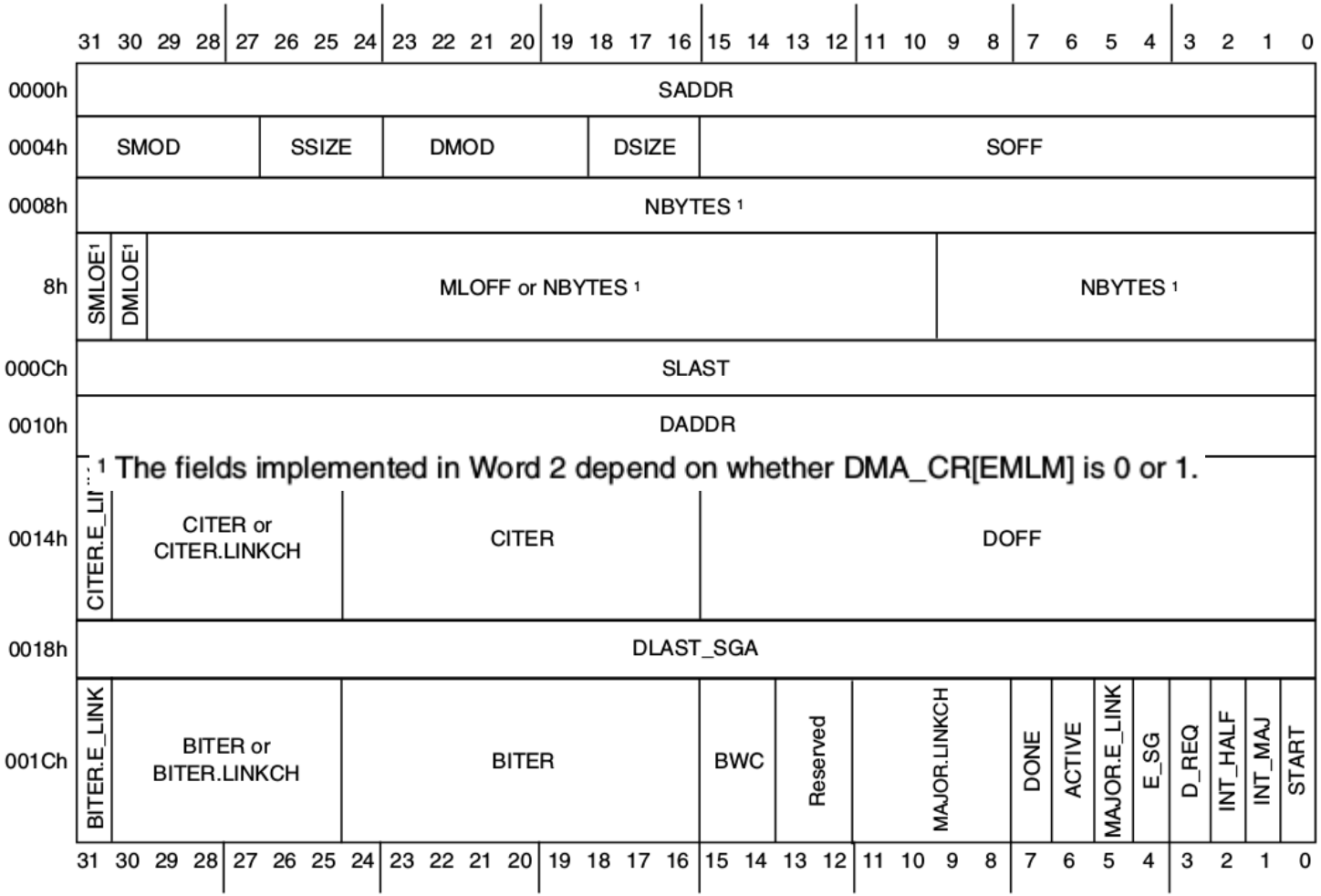
• La unidad de memoria **TCD** (Transfer control Descriptor memory) es una memoria local. Esa memoria se puede ver como un arreglo de estructuras del tipo TCD que contiene la información para la transferencia de datos. Cada elemento del arreglo representa a un canal del eDMA. Cada TCD tiene 32 Bytes es decir 8 palabras de 32 bits.

eDMA – TCD Memory Map

TCD Memory Map

eDMA Offset	TCDn Register Name	Abbreviation	Width (bits)
0x00	Source Address	TCDn_SADDR	32
0x04	Transfer Attributes	TCDn_ATTR	16
0x06	Signed Source Address Offset	TCDn_SOFF	16
0x08	Minor Byte Count	TCDn_NBYTES	32
0x0C	Last Source Address Adjustment	TCDn_SLAST	32
0x10	Destination Address	TCDn_DADDR	32
0x14	Current Minor Loop Link, Major Loop Count	TCDn_CITER	16
0x16	Signed Destination Address Offset	TCDn_DOFF	16
0x18	Last Destination Address Adjustment/Scatter Gather Address	TDDn_DLAST_SGA	32
0x1C	Beginning Minor Loop Link, Major Loop Count	TCDn_BITER	16
0x1E	Control and Status	TCDn_CSR	16

eDMA – TCD Memory Map



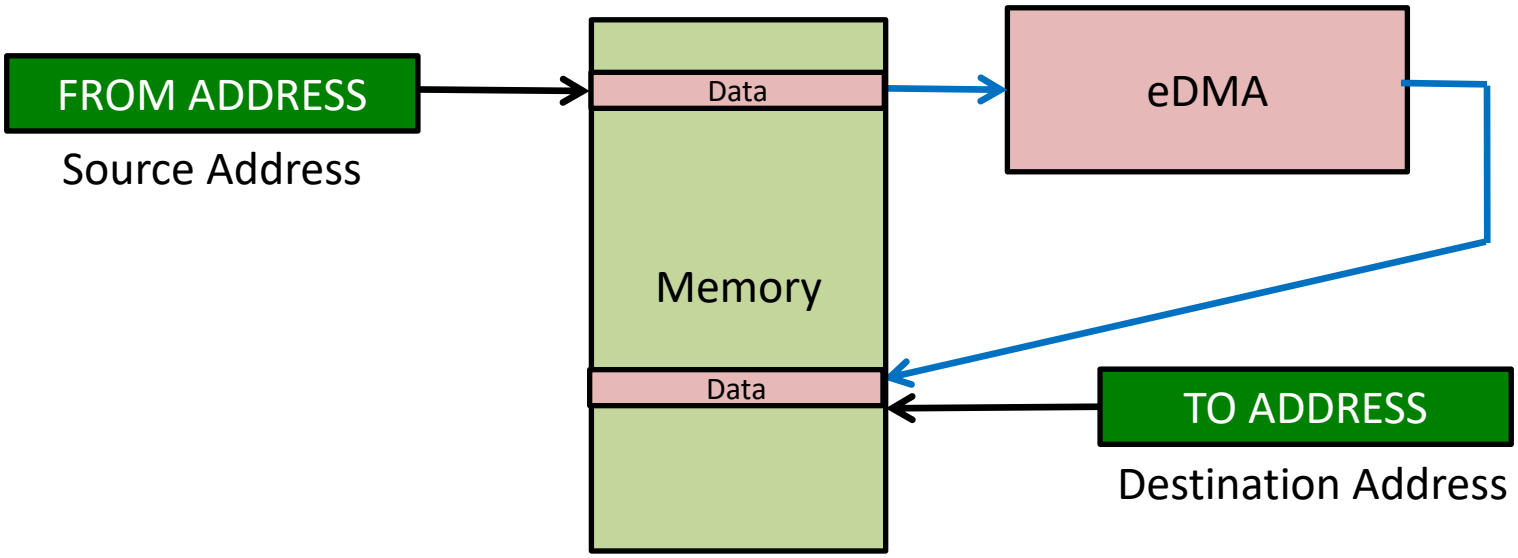
1 The fields implemented in Word 2 depend on whether DMA_CR[EMLM] is 0 or 1.

eDMA – TCD Memory Map

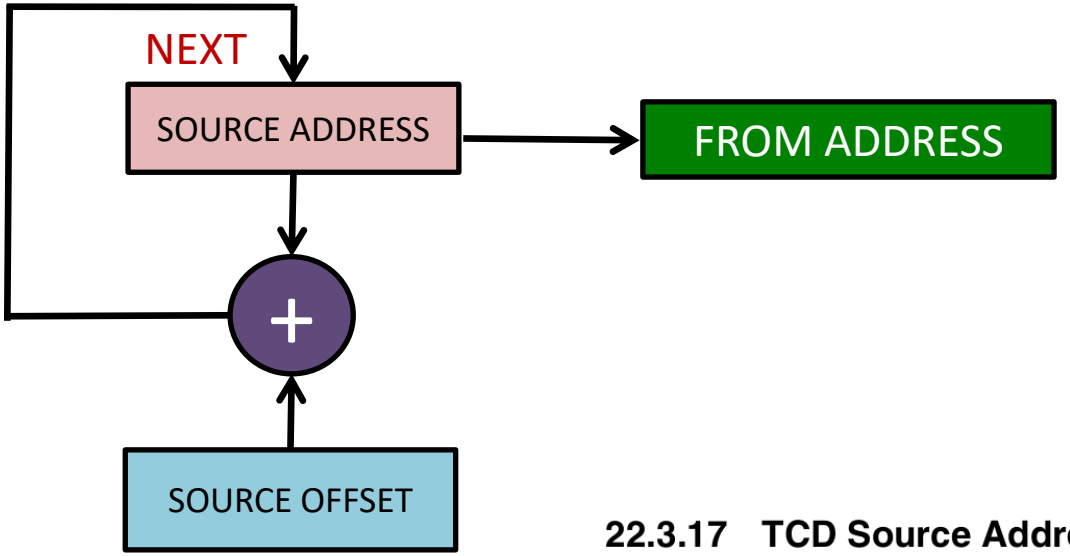
```
/* Structure with the TCD fields. */

typedef struct
{
        uint32_t SADDR;
        uint16_t SOFF;
        uint16_t ATTR;
        union
        {
                uint32_t NBYTES_MLNO;
                uint32_t NBYTES_MLOFFNO;
                uint32_t NBYTES_MLOFFYES;
        };
        uint32_t SLAST;
        uint32_t DADDR;
        uint16_t DOFF;
        union
        {
                uint16_t CITER_ELINKNO;
                uint16_t CITER_ELINKYES;
        };
        uint32_t DLASTSGA;
        uint16_t CSR;
        union
        {
                uint16_t BITER_ELINKNO;
                uint16_t BITER_ELINKYES;
        };
}TCD_t;
```

eDMA – Source - Destination



eDMA – Source - Destination



22.3.17 TCD Source Address (DMA_TCD*n*_SADDR)

Address: 4000_8000h base + 1000h offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R W	SADDR																															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

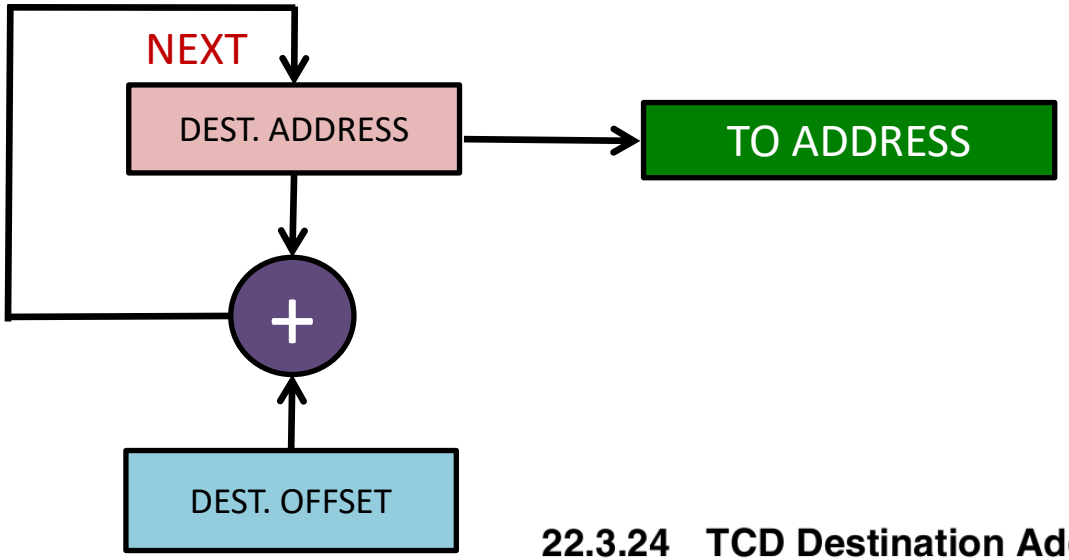
22.3.18 TCD Signed Source Address Offset (DMA_TCD*n*_SOFF)

Address: 4000_8000h base + 1004h offset + (32d × i), where i=0d to 15d

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read Write	SOFF															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

eDMA – Source - Destination



22.3.24 TCD Destination Address (DMA_TCD*n*_DADDR)

Address: 4000_8000h base + 1010h offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R W	DADDR																															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

22.3.25 TCD Signed Destination Address Offset (DMA_TCD*n*_DOFF)

Address: 4000_8000h base + 1014h offset + (32d × i), where i=0d to 15d

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read Write	DOFF															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

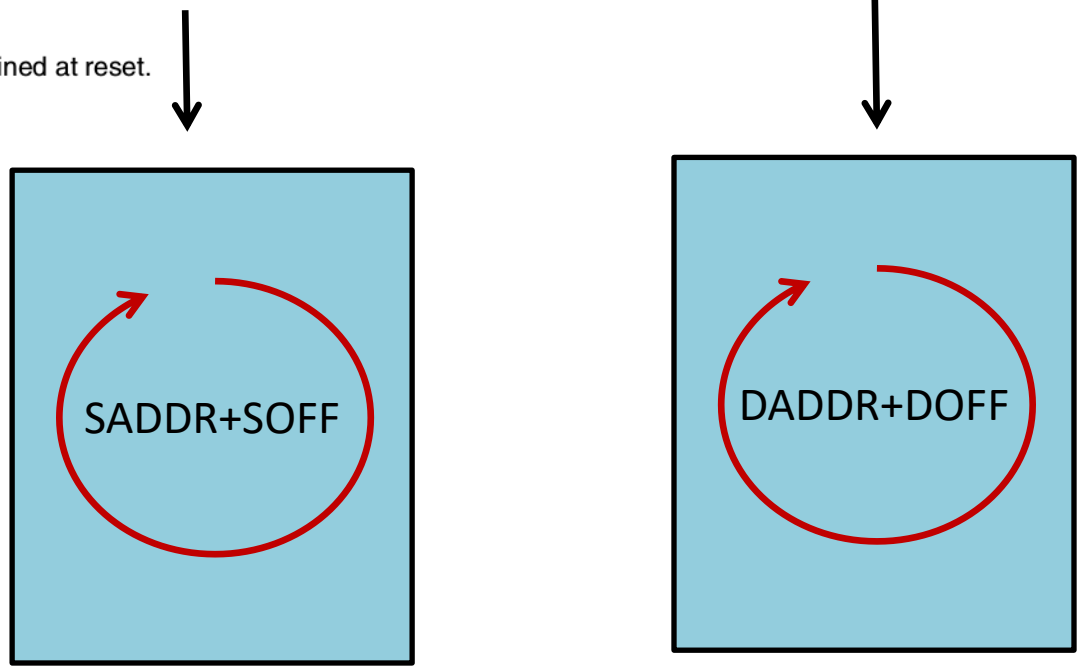
eDMA – Transfer Attributes

22.3.19 TCD Transfer Attributes (DMA_TCD*n*_ATTR)

Address: 4000_8000h base + 1006h offset + (32d × i), where i=0d to 15d

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read Write	SMOD					SSIZE			DMOD					DSIZE		
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.



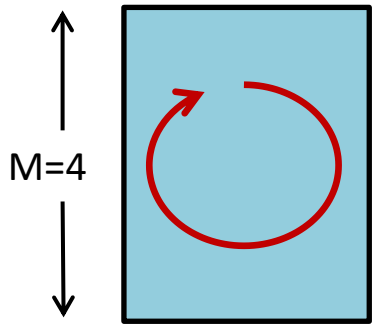
The value programmed into this field specifies the number of lower address bits allowed to change.

eDMA – Transfer Attributes: SMOD

SMOD=K Define un buffer circular => los K bits menos significativos de SADDR **deben** ser = 0. O sea que SADDR = Q . 2^K donde Q = 0,1,2,3,……..
De esta manera SADDR+SOFF queda confinado en este buffer circular de modulo M.

Ej: Si SMOD=2. K=2 => M=2^2=4. Buffer circular de 4 elementos. SADDR debe ser múltiplo de 2^2 .
Asi SADDR = Q 2^2 Q=0,1,2,3….

SADRR														
31	30	29	28	27	26	25	…	6	5	4	3	2	1	0
A	A	A	A	A	A	A	A	A	A	A	A	A	0	0



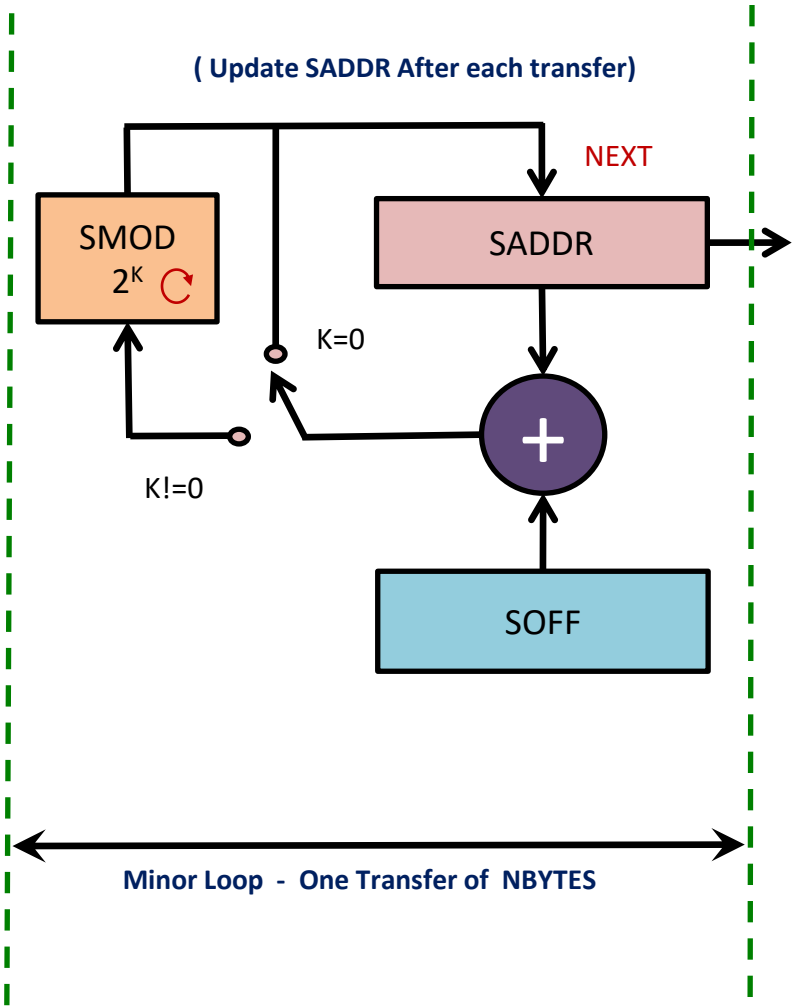
Nota: Esto aplica de la misma forma para DMOD

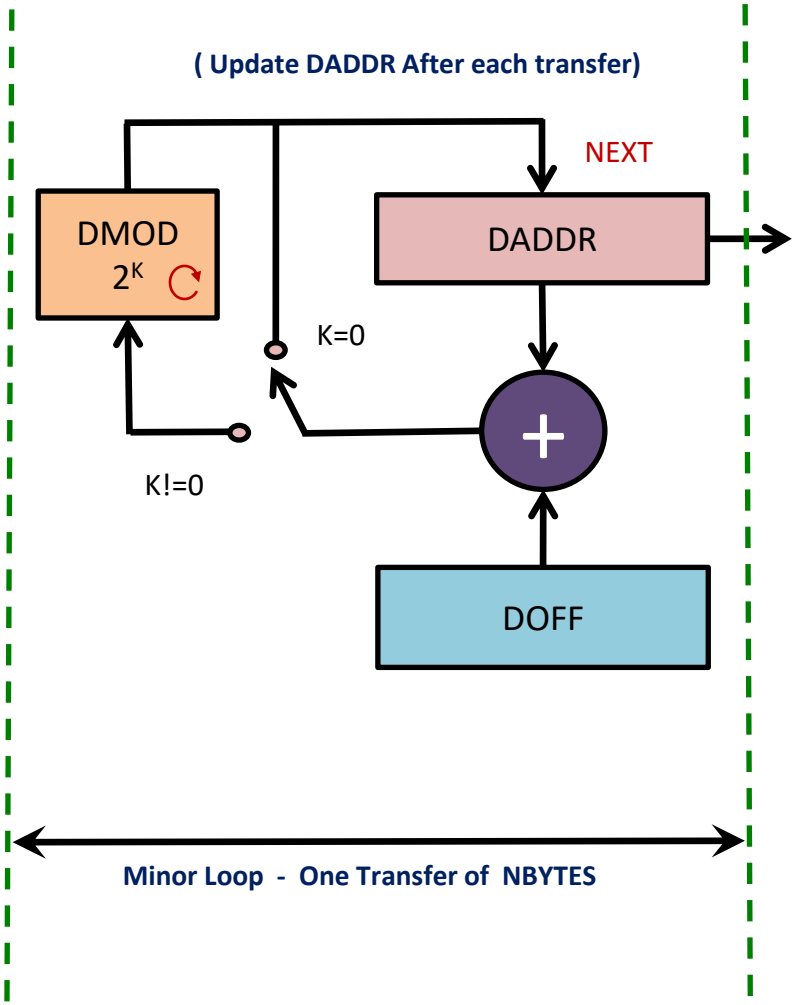
SADDR+[(SADDR + SOFF) **MOD** 2^K]→ SADDR

DMA_TCD0_ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_SMOD(2);

8 Bit Transfer | 8 Bit Transfer | K=2 Bit => Modulo 4

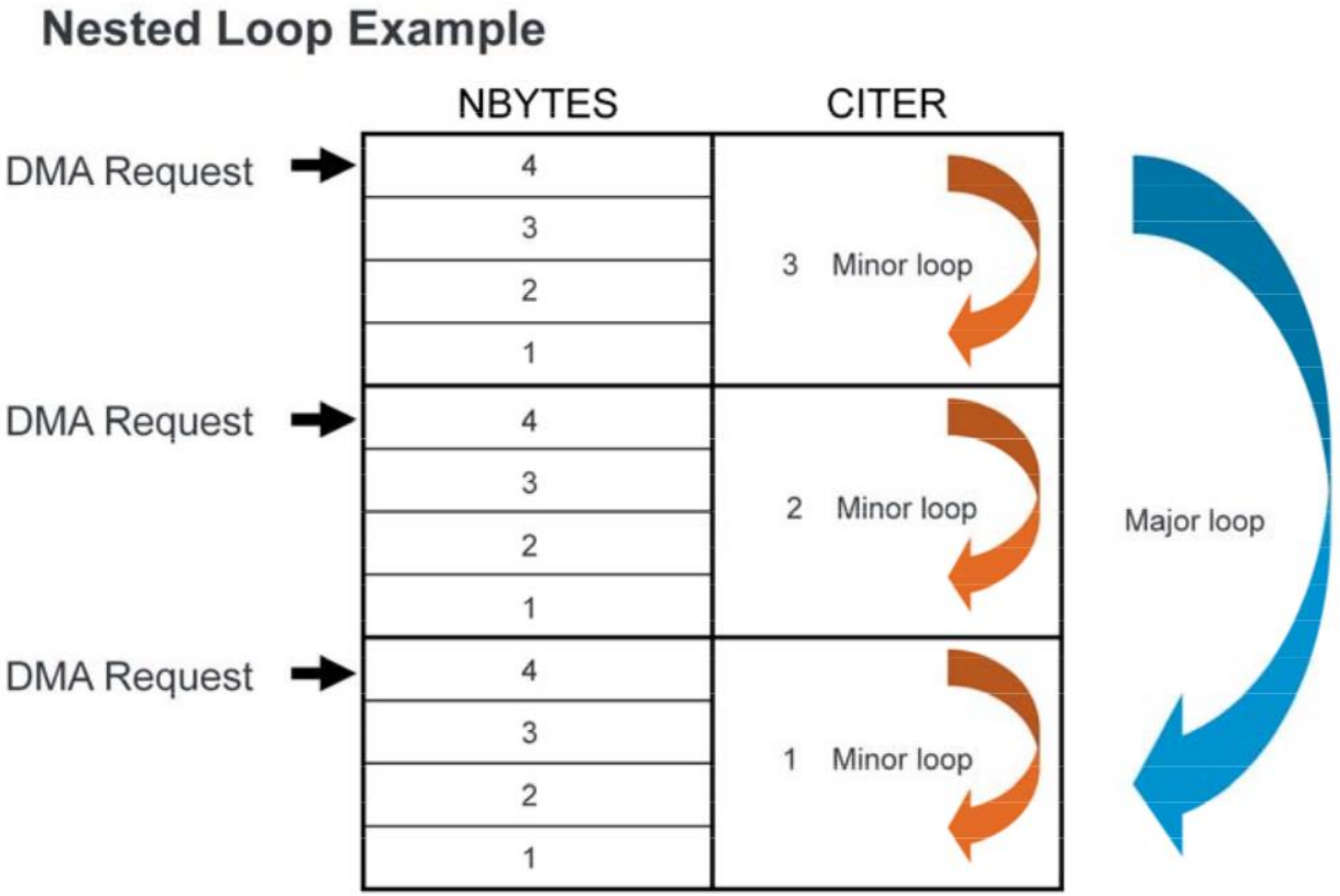
eDMA –Destination /Source Address Modulo



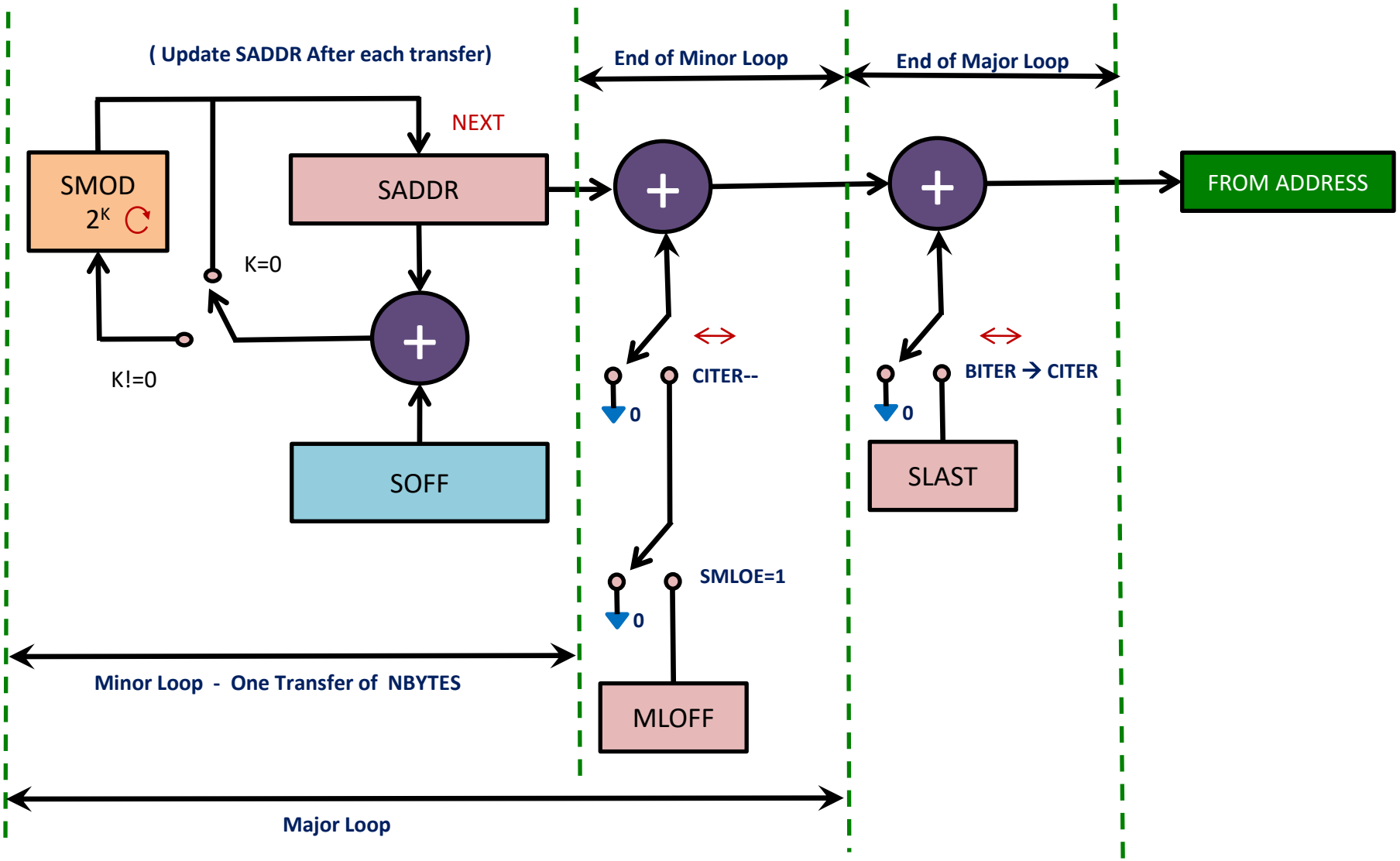


eDMA – TCD Minor and Major Byte Count

CITER: **Current Major Iteration Count**



eDMA –Source Address Generation Diagram



↔ Se conecta cuando se da la condición y se abre al iniciarse la siguiente iteración

eDMA – Transfer Attributes

22.3.19 TCD Transfer Attributes (DMA_TCD*n*_ATTR)

Address: 4000_8000h base + 1006h offset + (32d × i), where i=0d to 15d

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read Write	SMOD					SSIZE			DMOD					DSIZE		
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

NBYTES=K* XSIZE

000	8-bit
001	16-bit
010	32-bit
011	Reserved
100	16-byte
101	32-byte
110	Reserved
111	Reserved

000	8-bit
001	16-bit
010	32-bit
011	Reserved
100	16-byte
101	32-byte
110	Reserved
111	Reserved

Ejemplos:

```
DMA_0.TCD[0].SSIZE = 0x2; /* 32-bit */
DMA_0.TCD[0].DSIZE = 0x2; /* 32-bit */
DMA_0.TCD[0].NBYTES = 64; /* 16x32-bits */

DMA_0.TCD[0].SSIZE = 0x2; /* 32-bit */
DMA_0.TCD[0].DSIZE = 0x2; /* 32-bit */
DMA_0.TCD[1].NBYTES = 16; /* 4x32-bits */
```

Uso de Alineacion de variables para crear buffer circulares

Para crear un buffer circular de modulo $M=2^K$ se puede hacer uso del atributo **aligned**

En GCC (C99) es posible indicar al compilador que ubique una variable a partir de una posición de memoria cuya dirección sea el menor múltiplo posible de 2^n; es decir que la dirección de inicio será Q 2^n con Q=0,1,2,3….
Nótese que el valor de n representa el numero de bits menos significativos, que **deben** ser 0.
Una variable se puede alinear a un entorno de $L=2^n$ bytes usando el atributo **aligned** (alignment) de dicha variable. Sintaxis: **__attribute__ ((aligned (L))) .**

Ejemplos:

```
int x __attribute__ ((aligned (16))) = 0;  // n=4 ; 2^4=16=10000  L=16 Bytes
uint16_t sourceBuffer[10] __attribute__ ((aligned (16)))
={0x1234,0x6789,0x1122,0x2233,0x5588};
uint8_t  destinationBuffer[10] __attribute__ ((aligned (16))) ;
```

Los arreglos anteriores empiezan en una dirección que tiene sus 4 bits menos significativos en cero. Es decir que la dirección del arreglo **mod** L=0.

eDMA – Ejemplo Uso de DMOD 1/2

Ejemplo: Buffer circular de 4 bytes

```
uint16_t sourceBuffer[10] __attribute__ ((aligned (4))) ={0x1234,0x6789,0x1122,0x2233,0x5588};
uint8_t  destinationBuffer[10] __attribute__ ((aligned (4))) ;

/* Set an offset for source and destination address. */
DMA_TCD0_SOFF = 0x01; // Source address offset of 1 bytes per transaction.
DMA_TCD0_DOFF = 0x01; // Destination address offset of 1 byte per transaction.

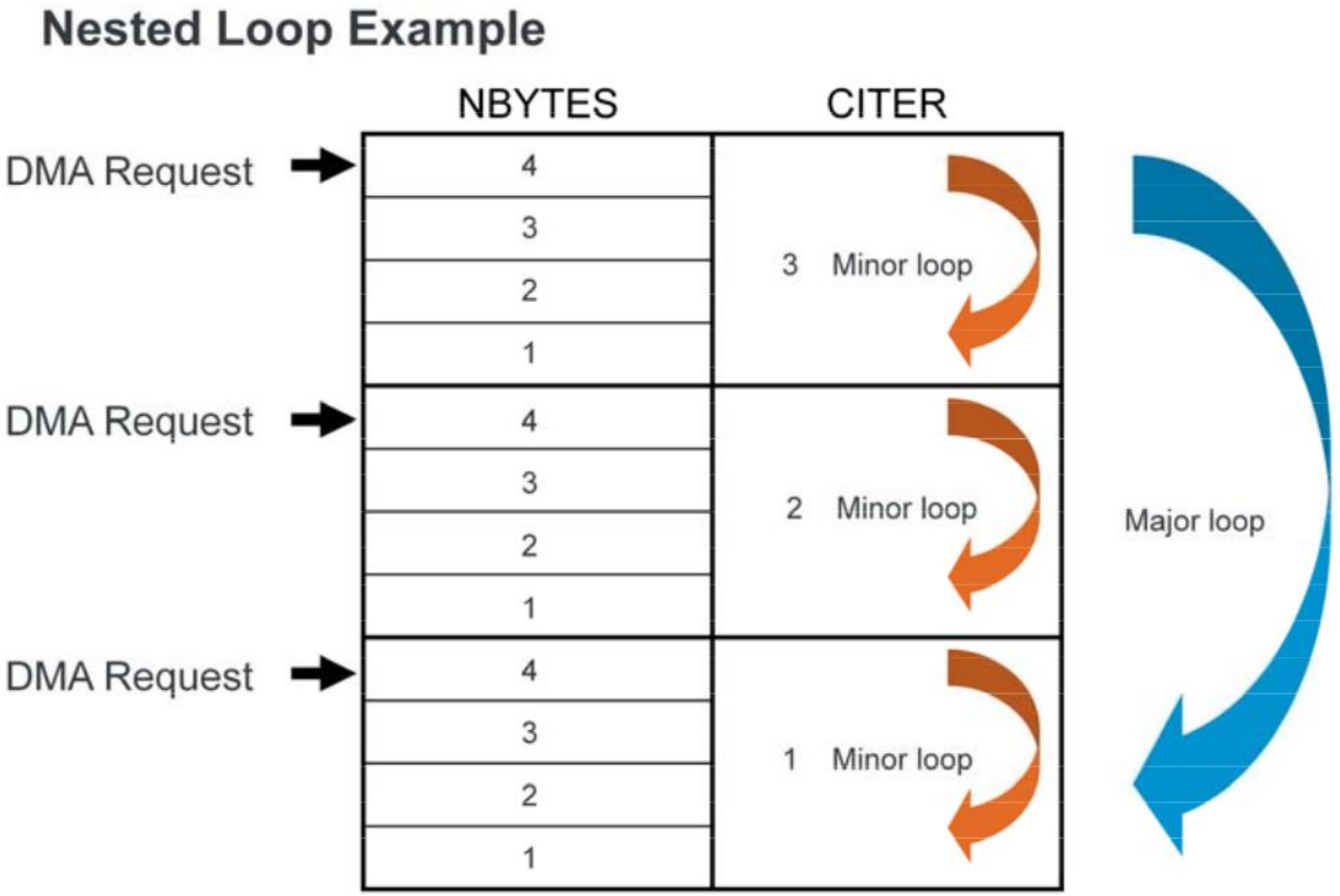
/* Set source and destination data transfer size is 1 byte. Modulo=4 */
DMA_TCD0_ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_DMOD(2);

/*Number of bytes to be transfered in each service request of the channel.*/
DMA_TCD0_NBYTES_MLNO = 0x02;

/* Current major iteration count (5 iteration of 1 byte each one). */
DMA_TCD0_CITER_ELINKNO = DMA_CITER_ELINKNO_CITER(0x05);
DMA_TCD0_BITER_ELINKNO = DMA_BITER_ELINKNO_BITER(0x05);
```

eDMA – TCD Minor and Major Byte Count

CITER: **Current Major Iteration Count**



eDMA– Ejemplo Uso de DMOD 2/2

Ejemplo: SOFF=1 DOFF=1 NBYTES=2 DZISE=SSIZE= 8 bit DMOD=2 (M=4) CITER=5

uint16_t **sourceBuffer**[10] = **__attribute__ ((aligned (4)))** {0x1234,0x6789,0x1122,0x2233,0x5588……}; DADDR=0x1FF009C

b1=b0=0

destinationBuffer	uint8_t [10]	0x1fff009c
destinationBuffer[0]	uint8_t	0x0
destinationBuffer[1]	uint8_t	0x0
destinationBuffer[2]	uint8_t	0x0
destinationBuffer[3]	uint8_t	0x0
destinationBuffer[4]	uint8_t	0x0

TCD0_DADDR	0x1FFF009C
TCD0_SADDR	0x1FFF0000
TCD0_CITER_ELINKYES	0x0005

destinationBuffer	uint8_t [10]	0x1fff009c
destinationBuffer[0]	uint8_t	0x34
destinationBuffer[1]	uint8_t	0x12
destinationBuffer[2]	uint8_t	0x0
destinationBuffer[3]	uint8_t	0x0
destinationBuffer[4]	uint8_t	0x0

TCD0_DADDR	0x1FFF009E
TCD0_SADDR	0x1FFF0002
TCD0_CITER_ELINKYES	0x0004

destinationBuffer	uint8_t [10]	0x1fff009c
destinationBuffer[0]	uint8_t	0x34
destinationBuffer[1]	uint8_t	0x12
destinationBuffer[2]	uint8_t	0x89
destinationBuffer[3]	uint8_t	0x67
destinationBuffer[4]	uint8_t	0x0

TCD0_DADDR	0x1FFF009C
TCD0_SADDR	0x1FFF0004
TCD0_CITER_ELINKYES	0x0003

Wrap

destinationBuffer	uint8_t [10]	0x1fff009c
destinationBuffer[0]	uint8_t	0x22
destinationBuffer[1]	uint8_t	0x11
destinationBuffer[2]	uint8_t	0x89
destinationBuffer[3]	uint8_t	0x67
destinationBuffer[4]	uint8_t	0x0

TCD0_DADDR	0x1FFF009E
TCD0_SADDR	0x1FFF0006
TCD0_CITER_ELINKYES	0x0002

destinationBuffer	uint8_t [10]	0x1fff009c
destinationBuffer[0]	uint8_t	0x22
destinationBuffer[1]	uint8_t	0x11
destinationBuffer[2]	uint8_t	0x33
destinationBuffer[3]	uint8_t	0x22
destinationBuffer[4]	uint8_t	0x0

TCD0_DADDR	0x1FFF009C
TCD0_SADDR	0x1FFF0008
TCD0_CITER_ELINKYES	0x0001

Wrap

eDMA – TCD Minor Byte Count

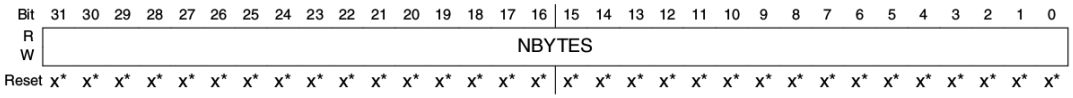
Enable Minor Loop Mapping

0 Disabled. TCDn.word2 is defined as a 32-bit NBYTES field.

1 Enabled. TCDn.word2 is redefined to include individual enable fields, an offset field, and the NBYTES field. The individual enable fields allow the minor loop offset to be applied to the source address, the destination address, or both. The NBYTES field is reduced when either offset is enabled.

Minor loop mapping is disabled (CR[EMLM] = 0)

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d

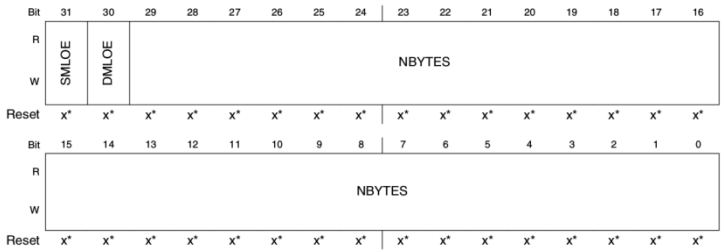


* Notes:
- x = Undefined at reset.

DMA_TCDn_NBYTES_MLNO

Minor loop mapping is enabled (CR[EMLM] = 1) and Minor loop offset is disabled (SMLOE=0 and DMLOE =0)

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d



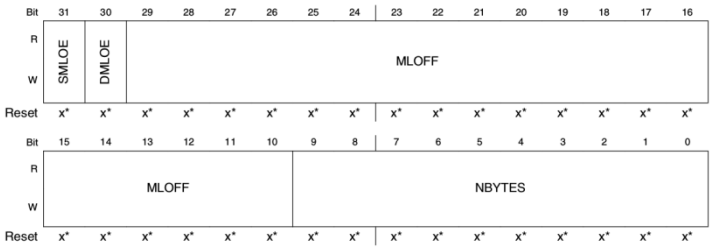
* Notes:
- x = Undefined at reset.

DMA_TCDn_NBYTES_MLOFFNO

```
union
{
        uint32_t NBYTES_MLNO;
        uint32_t NBYTES_MLOFFNO;
        uint32_t NBYTES_MLOFFYES;
};
```

Minor loop mapping is enabled (CR[EMLM] = 1) and Minor loop offset is enabled (SMLOE or DMLOE = 1)

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d



* Notes:
- x = Undefined at reset.

DMA_TCDn_NBYTES_MLOFFYES

MLOFF= Minor Loop Offset
SMLOE= Source Minor Loop Offset Enable
DMLOE=Destination Minor Loop Offset Enable

eDMA – TCD Minor Byte Count

```
union
{
        uint32_t NBYTES_MLNO;
        uint32_t NBYTES_MLOFFNO;
        uint32_t NBYTES_MLOFFYES;
};
```

Minor loop mapping is disabled (CR[EMLM] = 0)

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d

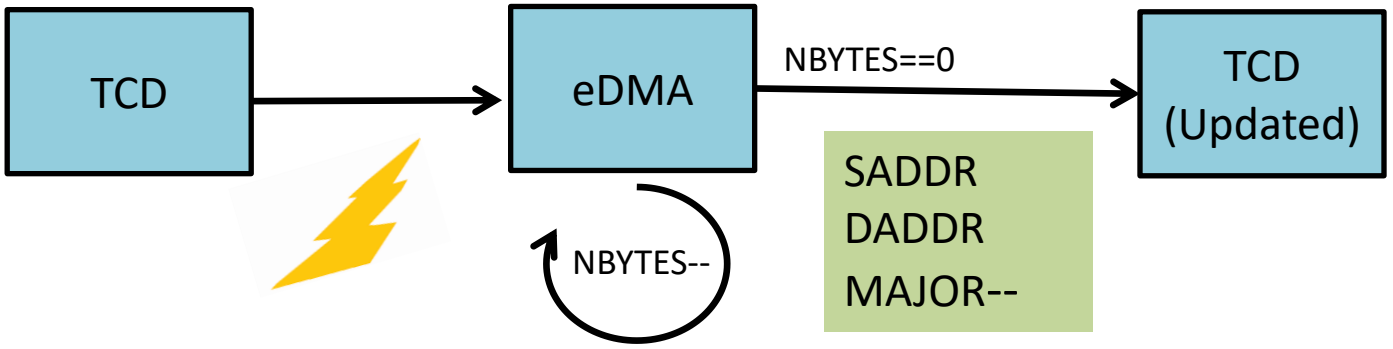
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R W	NBYTES																															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

Minor Byte Transfer Count

DMA_TCDn_NBYTES_MLNO

Number of bytes to be transferred in **each service request** of the channel
This is an indivisible operation and cannot be halted.



DMA service request

NOTE: An NBYTES value of 0x0000_0000 is interpreted as a 4 GB transfer.

eDMA – TCD Minor Byte Count

Minor loop mapping is enabled (CR[EMLM] = 1) and Minor loop offset is disabled (SMLOE=0 and DMLOE =0)

DMA_TCDn_NBYTES_MLOFFNO

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	SMLOE	DMLOE	NBYTES													
W	0	0														
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	NBYTES															
W																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

En este caso **NO** se aplica el offset !!!!

```
union
{
        uint32_t NBYTES_MLNO;
        uint32_t NBYTES_MLOFFNO;
        uint32_t NBYTES_MLOFFYES;
};
```

eDMA – TCD Minor Byte Count

Minor loop mapping is enabled (CR[EMLM] = 1) and Minor loop offset is enabled (SMLOE or DMLOE = 1)

DMA_TCDn_NBYTES_MLOFFYES

Address: 4000_8000h base + 1008h offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	SMLOE	DMLOE	MLOFF													
W	!=0	!=0														
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MLOFF								NBYTES							
W																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

En este caso **SI** se aplica el offset !!!!

MLOFF + SADD si SMLOE=1
MLOFF + DADD si DMLOE=1

MLOFF: Minor Loop Offset

MLOFF se aplica cada vez que finaliza el minor loop

```
union
{
        uint32_t NBYTES_MLNO;
        uint32_t NBYTES_MLOFFNO;
        uint32_t NBYTES_MLOFFYES;
};
```

eDMA – TCD Last Source Address Adjustment

TCD Last Source Address Adjustment

Address: 4000_8000h base + 100Ch offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R W	SLAST																															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:
- x = Undefined at reset.

DMA_TCD*n*_SLAST

Valor que se le agrega al source address después de una iteración mayor

Puede ser usado para pasar a la próxima estructura de datos o volver al principio ya que es **signado**

eDMA – TCD Last Destination Address Adjustment/Scatter Gather Address

Address: 4000_8000h base + 1018h offset + (32d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R W	DLASTSGA																															
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

DMA_TCDn_DLASTSGA

* Notes:
- x = Undefined at reset.

Si TCDn_CSR[ESG] = 0 este registro se comporta igual que DMA_TCDn_SLAST pero con Destination Address

Si TCDn_CSR[ESG] = 1 esta dirección hace referencia al principio de una región modular de 32 bytes que contiene el próximo TCD que será cargado en este canal.
CSR = Control Status Register ESG = Enable Scatter and Gather

Scatter/gather feature

As it is stated in the Reference Manual, scatter/gather is the process of automatically loading a new TCD into a channel, so it allows an eDMA channel to use multiple TCDs. The name is derived from the case to enable an eDMA channel to **scatter** the eDMA data to multiple destinations or **gather** it from multiple sources.

When scatter/gather is enabled and the channel has finished its major loop, a new TCD is fetched from system memory and loaded into that channel's descriptor location in eDMA programmer's model, thus replacing the current descriptor.

Scatter and Gather

```c
/* Structure with the TCD fields. */
typedef struct
{
    uint32_t SADDR;
    uint16_t SOFF;
    uint16_t ATTR;
    union
    {
        uint32_t NBYTES_MLNO;
        uint32_t NBYTES_MLOFFNO;
        uint32_t NBYTES_MLOFFYES;
    };
    uint32_t SLAST;
    uint32_t DADDR;
    uint16_t DOFF;
    union
    {
        uint16_t CITER_ELINKNO;
        uint16_t CITER_ELINKYES;
    };
    uint32_t DLASTSGA;
    uint16_t CSR;
    union
    {
        uint16_t BITER_ELINKNO;
        uint16_t BITER_ELINKYES;
    };
}TCD_t;
```

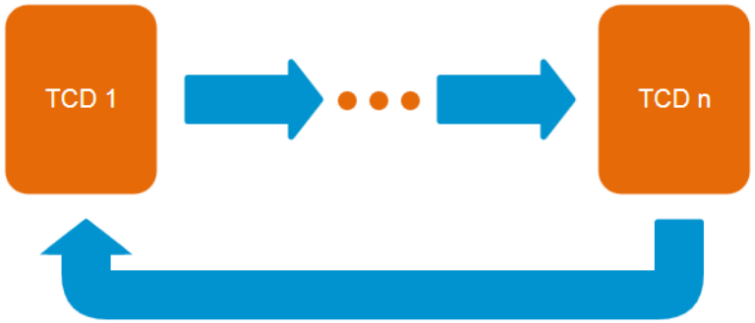
```c
/* Enumeration to access the TCDs. */
enum{
        TCD0 = 0,
        TCD1,
        TCDs_AMOUNT
}TCD_enum;
```

```c
/* Declare the two TCDs that will be used, these must be memory aligned. */
TCD_t tcd[TCDs_AMOUNT] __attribute__ ((aligned (32)));

tcd[TCD0].DLASTSGA = (uint32_t)&tcd[TCD1];

tcd[TCD1].DLASTSGA = (uint32_t)&tcd[TCD0];

DMA_TCD0_DLASTSGA = tcd[TCD0].DLASTSGA;
```



eDMA – TCD Current Minor Loop Link, Major Loop Count

```
union
{
        uint16_t CITER_ELINKNO;
        uint16_t CITER_ELINKYES;
};
```

TCDn_CITER si TCDn_CITER[ELINK]=0

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	CITER						
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	CITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

TCDn_CITER si TCDn_CITER[ELINK]=1

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	0			LINKCH			CITER
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	CITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

CITER: Current Major Iteration Count

eDMA – TCD Current Minor Loop Link, Major Loop Count

TCDn_CITER si TCDn_CITER[ELINK]=0

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	CITER						
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	CITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

ELINK: Enables channel-to-channel linking on minor loop complete.

CITER: Current Major Iteration Count . count represents the current major loop count for the channel. It is decremented each time the minor loop is completed and updated in the transfer control descriptor memory. After the major iteration count is exhausted, the channel performs a number of operations (e.g., final source and destination address calculations), optionally generating an interrupt to signal channel completion before reloading the CITER field from the beginning iteration count (BITER) field.
CITER: 9-bit (ELINK = 1) or 15-bit (ELINK = 0)

BITER → CITER Reload CITER from BITER When CITER == 0

NOTE: When the CITER field is initially loaded by software, it must be set to the same value as that contained in the BITER field.
NOTE: If the channel is configured to execute a single service request, the initial values of BITER and CITER should be 0x0001.

eDMA – TCD Control and Status (DMA_TCDn_CSR)

Bit	15	14	13	12	11	10	9	8
Read / Write	BWC		0		MAJORLINKCH			
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	DONE	ACTIVE	MAJORELINK	ESG	DREQ	INTHALF	INTMAJOR	START
Reset	0	0	x*	x*	x*	x*	x*	0

BWC: Band width control: none 4 or 8 cpu cycles stall.

MAJORLINKCH: Channel Number for Channel-to-channel linking (or chaining) is performed after the major loop counter is exhausted.

MAJORELINK: Enable/Disable Channel-to-channel linking on major loop complete.

DONE: This flag indicates the eDMA has completed the major loop. CITER → 0

ACTIVE: This flag signals the channel is currently in execution. eDMA clears it as the minor loop completes

ESG: Enable Scatter/Gather Processing

DREQ: Disable Request

INTHALF (INTMAJOR): Enables interrupt when CITER reaches the halffway count (when CITER reaches 0).

START: The channel is explicitly started via a software initiated service request

eDMA – TCD Beginning Minor Loop Link, Major Loop Count

TCDn_BITER[ELINK]=0

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	BITER						
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	BITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

TCDn_BITER[ELINK]=1

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	0			LINKCH			BITER
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	BITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

```
union
{
        uint16_t BITER_ELINKNO;
        uint16_t BITER_ELINKYES;
};
```

BITER: Start Current Major Iteration Count

eDMA – TCD Beginning Minor Loop Link, Major Loop Count

TCDn_BITER[ELINK]=0

Bit	15	14	13	12	11	10	9	8
Read / Write	ELINK	BITER						
Reset	x*	x*	x*	x*	x*	x*	x*	x*

Bit	7	6	5	4	3	2	1	0
Read / Write	BITER							
Reset	x*	x*	x*	x*	x*	x*	x*	x*

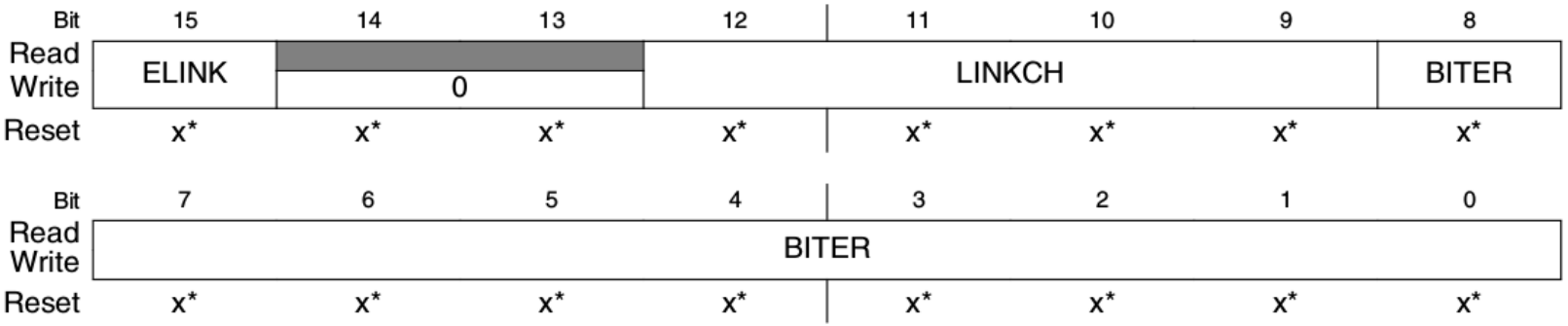
ELINK: Enables channel-to-channel linking on minor loop complete.

BITER: Starting Major Iteration Count (ojo debe ser igual a CITER cuando se inicializa por primera vez la TCD) Cada vez que termina una iteración mayor BITER → CITER

BITER: Start Current Major Iteration Count

eDMA – TCD Beginning Minor Loop Link, Major Loop Count

TCDn_BITER[ELINK]=1



ELINK: Enables channel-to-channel linking on minor loop complete.
LINKCH: Link Channel Number
BITER: Starting Major Iteration Count (ojo debe ser igual a CITER cuando se inicializa por primera vez la TCD) Cada vez que termina una iteración mayor BITER → CITER

BITER: Start Current Major Iteration Count

eDMA – Enable Request Register (DMA_ERQ)

Address: 4000_8000h base + Ch offset = 4000_800Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ERQ15	ERQ14	ERQ13	ERQ12	ERQ11	ERQ10	ERQ9	ERQ8	ERQ7	ERQ6	ERQ5	ERQ4	ERQ3	ERQ2	ERQ1	ERQ0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

DMA_ERQ field descriptions

Enable the request signal for each channel.

ERQx: Enable =1 Disable =0

Set Enable Request Register (DMA_SERQ)

Set bit to enable the request signal for each channel.

Address: 4000_8000h base + 1Bh offset = 4000_801Bh

Bit	7	6	5	4	3	2	1	0
Read	0	0			0			
Write	NOP	SAER	0		SERQ			
Reset	0	0	0	0	0	0	0	0

DMA_SERQ field descriptions

NOP: (= 0) Normal Operation (This allows you to write multiple-byte registers as a 32-bit word.)
(= 1) No operation, ignore the other bits in this register

SAER: (= 0) Set only the ERQ bit specified in the SERQ field
(= 1) Set all bits in ERQ

SERQ: Set enable request. Sets the corresponding bit in ERQ

The {S,C}ERQ registers are provided so the request enable for a single channel can easily be modified without needing to perform a read-modify-write sequence to the ERQ.

Clear Enable Request Register (DMA_CERQ)

Clear bit to disable the request signal for each channel.

Address: 4000_8000h base + 1Ah offset = 4000_801Ah

Bit	7	6	5	4	3	2	1	0
Read	0	0			0			
Write	NOP	CAER	0		CERQ			
Reset	0	0	0	0	0	0	0	0

DMA_CERQ field descriptions

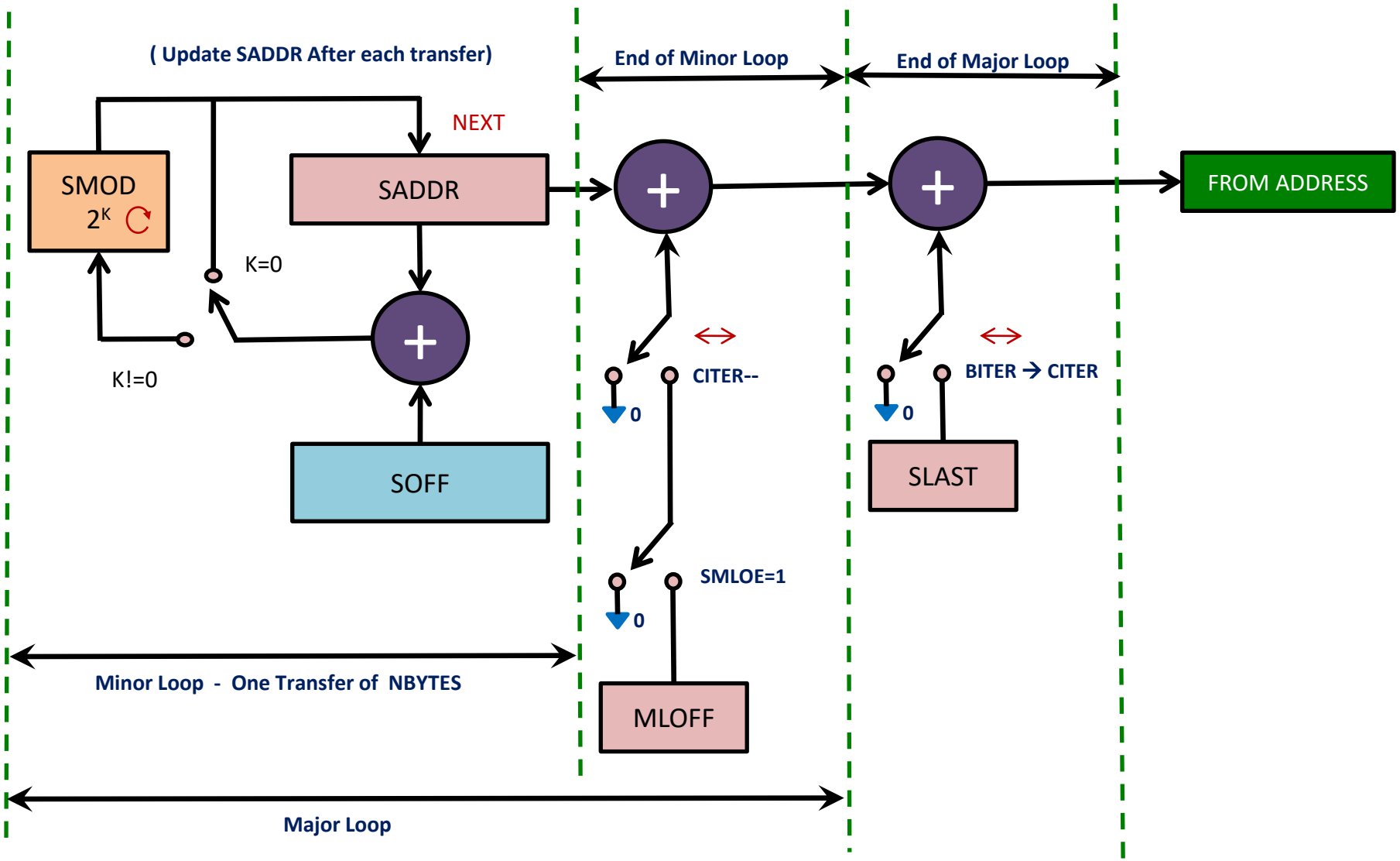
NOP: (= 0) Normal Operation (This allows you to write multiple-byte registers as a 32-bit word.)
(= 1) No operation, ignore the other bits in this register

CAER: (= 0) Clear only the ERQ bit specified in the SERQ field
(= 1) Clear all bits in ERQ

SERQ: Clear enable request. Clear the corresponding bit in ERQ

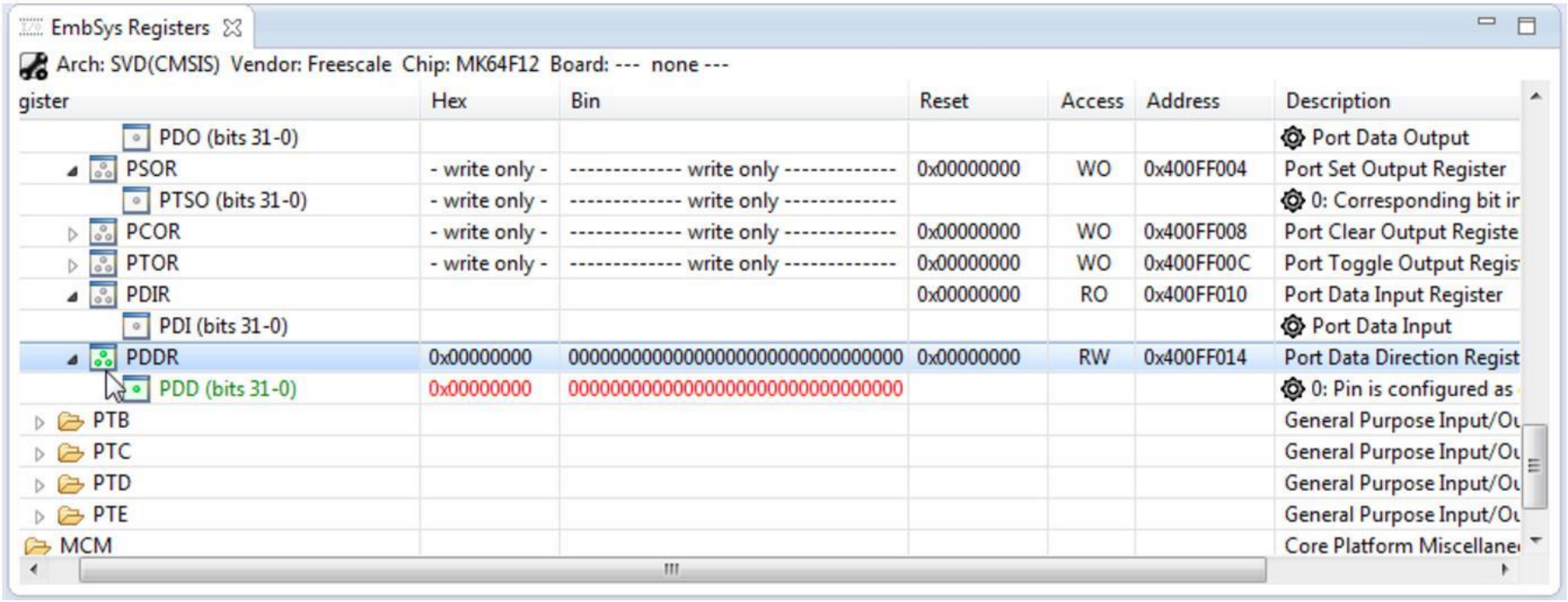
The {S,C}ERQ registers are provided so the request enable for a single channel can easily be modified without needing to perform a read-modify-write sequence to the ERQ.

eDMA –Source Address Generation Diagram



↔ Se conecta cuando se da la condición y se abre al iniciarse la siguiente iteración

Useful Tool for DMA debug & Learn: EmbSysRegView



Embedded Systems Register View

END Part1

Q & A:

Q: Is there a DMA channel available to transfer my groceries from supermarket to home ?. I'm so lazy.

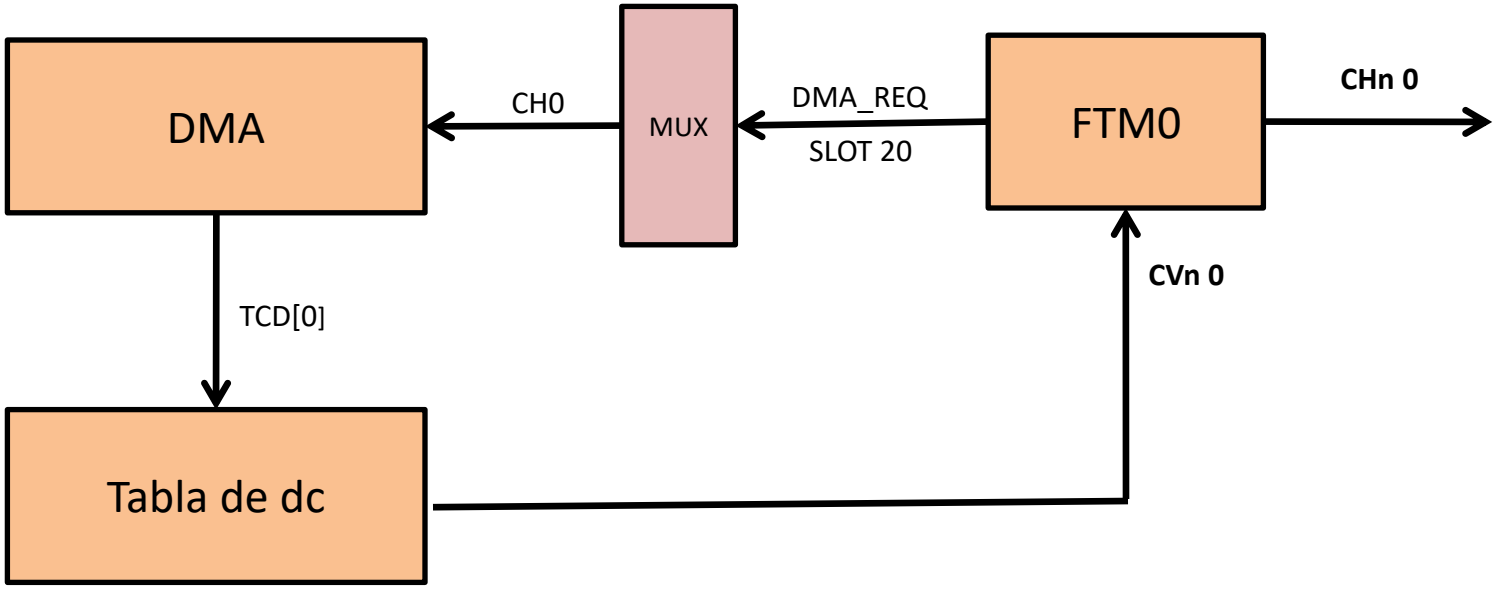
A: NO !!!!

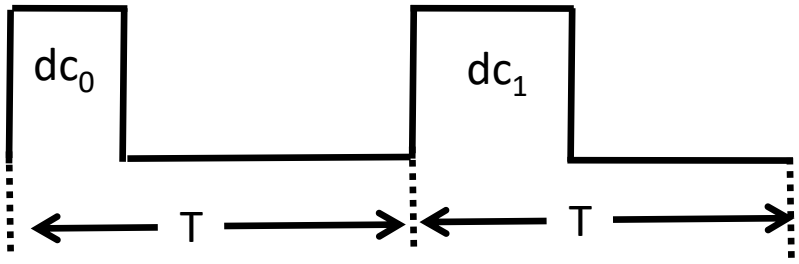
References :

- **K64 Sub-Family Reference Manual, Rev. 2, January 2014.**
- **Using the DMA module in Kinetis devices - Freescale Semiconductor.**
- **How to Add Register Details View in Eclipse (The EmbSysRegView project)**
  https://mcuoneclipse.com/2014/05/29/how-to-add-register-details-view-in-eclipse/

Example DMA- FTM

Generar una señal PWM a partir de una tabla en memoria





uint16_t sourceBuffer[] = {1000 ,2000,3000,4000,5000,6000,7000,8000};

Example DMA- FTM

TCD0 (Transfer control descriptor)

```
/* Set memory address for source and destination. */
DMA_TCD0_SADDR = (uint32_t)(sourceBuffer);   //List of Duties
DMA_TCD0_DADDR = (uint32_t)(&(FTM0->CONTROLS[0].CnV));  // To change FTM Duty

/* Set an offset for source and destination address. */
DMA_TCD0_SOFF =0x02; // Source address offset of 2 bytes per transaction.
DMA_TCD0_DOFF =0x00; // Destination address offset of 0 byte per transaction. (Siempre al mismo lugar)

/* Set source and destination data transfer size is 1 byte. */
DMA_TCD0_ATTR = DMA_ATTR_SSIZE(1) | DMA_ATTR_DSIZE(1);

/*Number of bytes to be transfered in each service request of the channel.*/
DMA_TCD0_NBYTES_MLNO = 0x02;

/* Autosize for Current major iteration count */
DMA_TCD0_CITER_ELINKNO = DMA_CITER_ELINKNO_CITER((sizeof(sourceBuffer)/sizeof(sourceBuffer[0])));
DMA_TCD0_BITER_ELINKNO = DMA_BITER_ELINKNO_BITER((sizeof(sourceBuffer)/sizeof(sourceBuffer[0])));

/* Autosize SLAST for Wrap Around. This value is added to SADD at the end of Major Loop */
DMA_TCD0_SLAST = -sizeof(sourceBuffer);

 /* DLASTSGA DLAST Scatter and Gatter (Not Used) */
 DMA_TCD0_DLASTSGA = 0x00;
```

Example DMA- FTM

FTM0

40.4.23 DMA

The channel generates a DMA transfer request according to DMA and CHnIE bits. See the following table.

Table 40-311. Channel DMA transfer request

DMA	CHnIE	Channel DMA Transfer Request	Channel Interrupt
0	0	The channel DMA transfer request is not generated.	The channel interrupt is not generated.
0	1	The channel DMA transfer request is not generated.	The channel interrupt is generated if (CHnF = 1).
1	0	The channel DMA transfer request is not generated.	The channel interrupt is not generated.
1	1	The channel DMA transfer request is generated if (CHnF = 1).	The channel interrupt is not generated.

If DMA = 1, the CHnF bit is cleared either by channel DMA transfer done or reading CnSC while CHnF is set and then writing a zero to CHnF bit according to CHnIE bit. See the following table.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0								CHF	CHIE	MSB	MSA	ELSB	ELSA	0	DMA
W									0							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

FTMx_CnSC field descriptions

Example DMA- FTM

FTM0

```
uint16_t PWM_modulus = 10000-1;
uint16_t PWM_duty    = 5000;

FTM_SetPrescaler(FTM0, FTM_PSC_x32);              // Prescaler x32

FTM_SetInterruptMode (FTM0,FTM_CH_0, true);  // Enable interrupts

FTM_SetWorkingMode(FTM0, 0, FTM_mPulseWidthModulation);      // MSA  / B

FTM_SetPulseWidthModulationLogic(FTM0, 0, FTM_lAssertedHigh);   // ELSA / B

FTM_SetModulus(FTM0, PWM_modulus);    // Modulo

FTM_SetCounter(FTM0, 0, PWM_duty);    // DC

FTM_DmaMode (FTM0,FTM_CH_0,FTM_DMA_ON); // DMA ON

FTM_StartClock(FTM0);       //Enable Clock
```

Example DMA- FTM

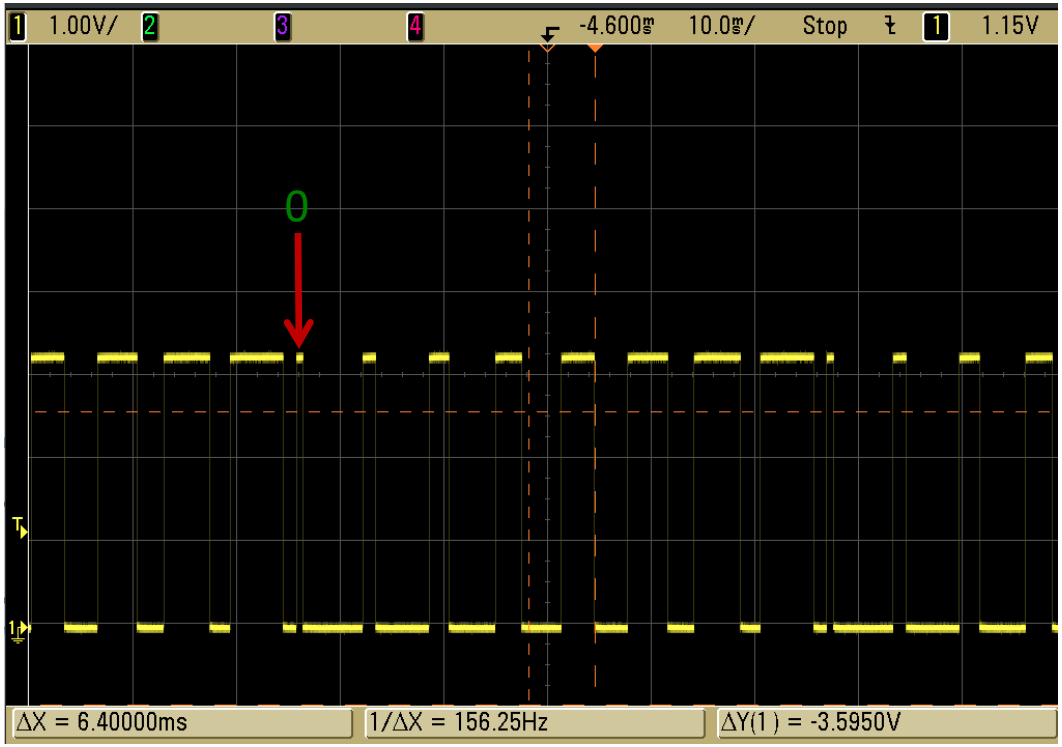
FTM0

Prescaler: 32x FTMclock: 50MHz T = 1/(50E6) X 32 X PWM_modulus = 6.40ms

uint16_t sourceBuffer[] = {1000 ,2000,3000,4000,5000,6000,7000,8000};

DTY=(1/50e6)*32* sourceBuffer[i]

0-1000-640 us
1-2000-1.28 ms
2-3000-1.92 ms
3-4000-2.56 ms
4-5000-3.20 ms
5-6000-3.84 ms
6-7000-4.48 ms
7-8000-5.12 ms

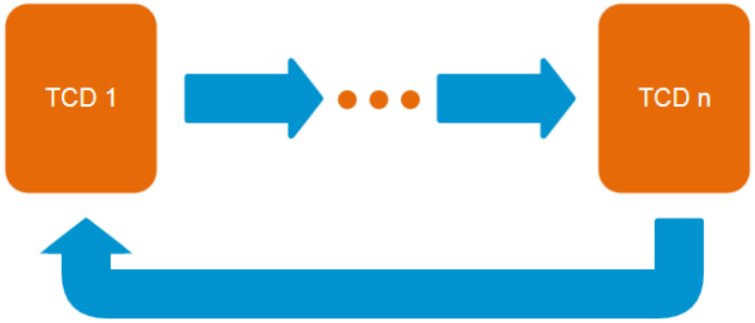


Advaced DMA Modes

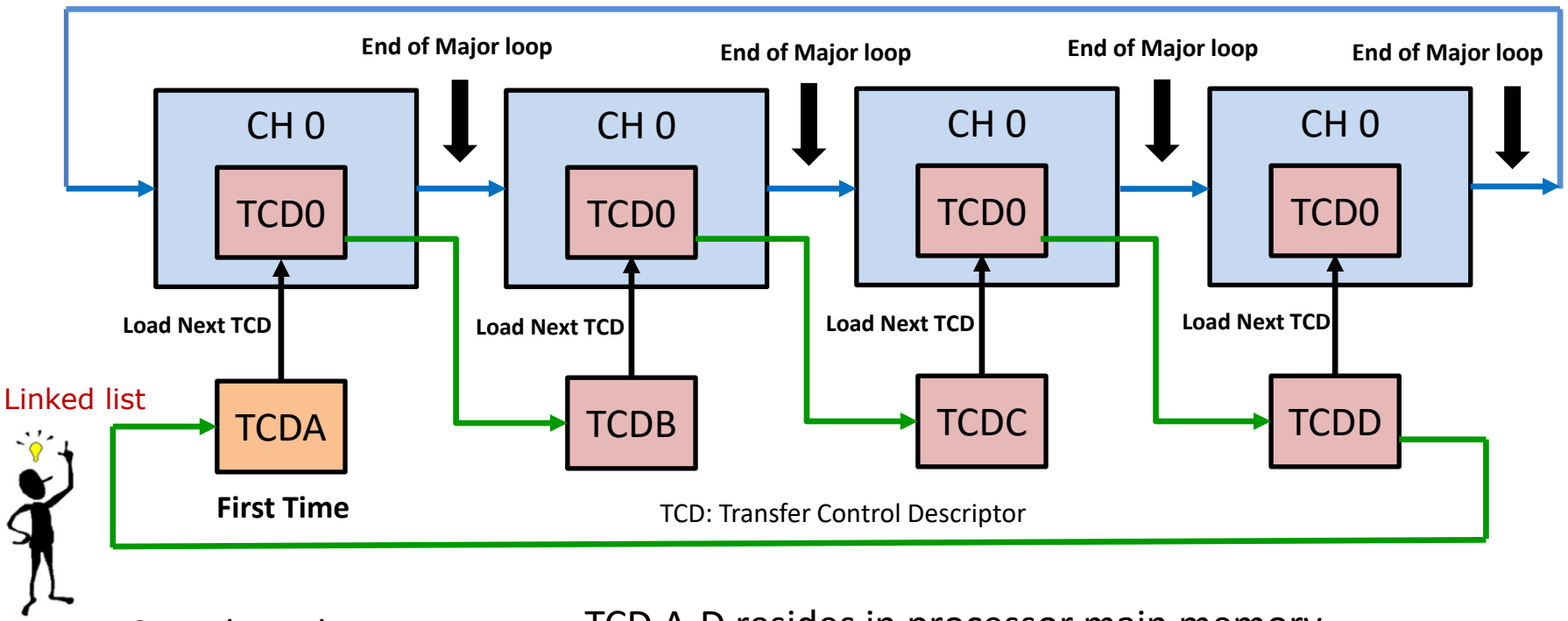
- Scatter & Gather
- Channel Linking

Scatter & Gather

- Scatter & Gather es el proceso que permite que un canal del DMA use múltiples TCD. Cada vez que finaliza el major loop un nuevo TCD es copiado al descriptor del canal, reemplazando así el descriptor actual.



Scatter & Gather



Sample code:

TCD A-D resides in processor main memory

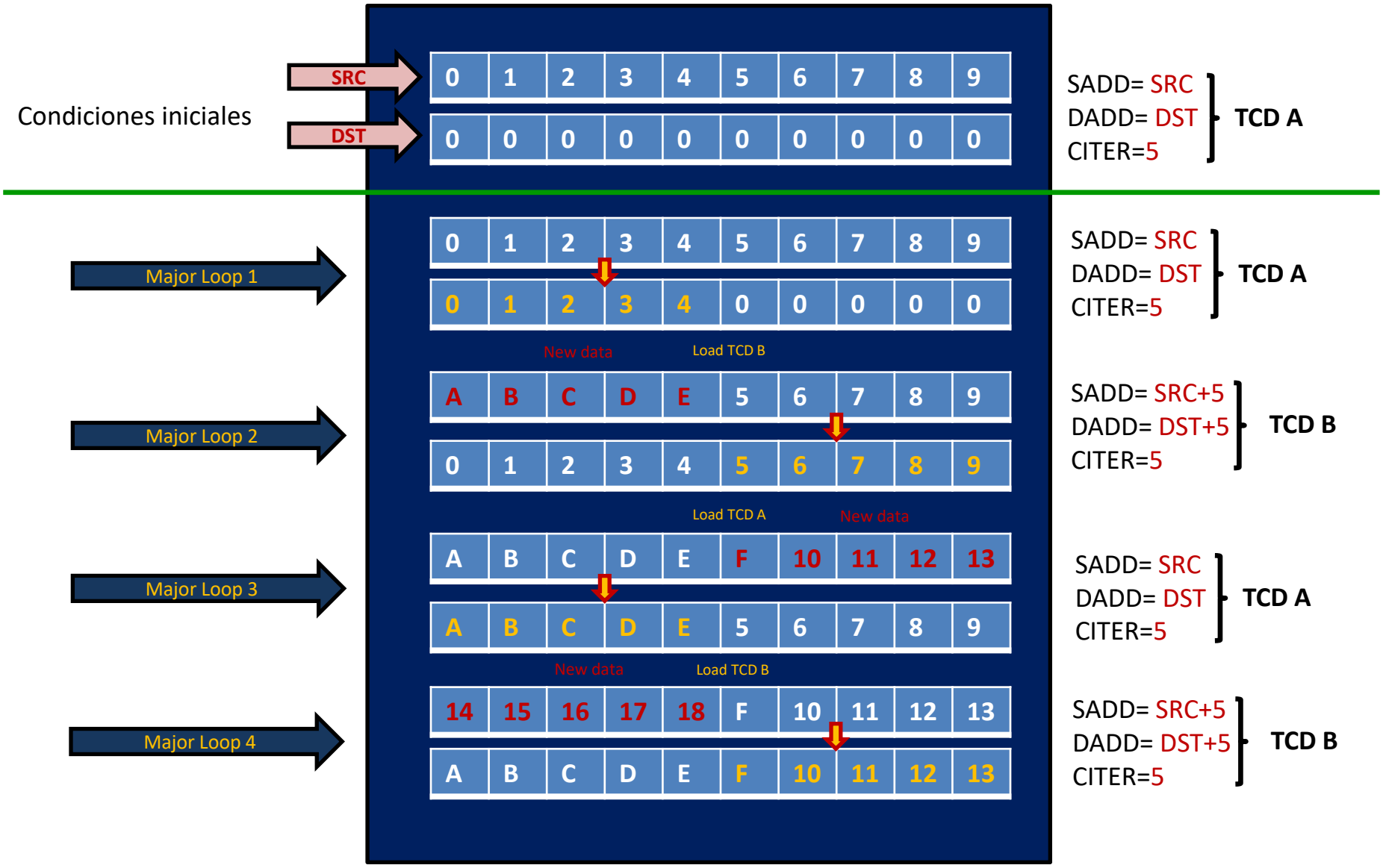
```
TCDA.DLASTSGA = (uint32_t)&TCDB;  // The TCDB is the next TCD to be loaded.
TCDB.DLASTSGA = (uint32_t)&TCDC;  // The TCDC is the next TCD to be loaded.
```

Ejemplo Buffer circular

- Este ejemplo esta basado en la nota de aplicación

  What is and how to configure the eDMA scatter-gather feature
- En este ejemplo se muestra como implementar un buffer circular que no sea múltiplo de 2^n.
- Se tienen 2 buffers circulares de 10 elementos (src,dst).
- El DMA copia datos de un buffer a otro en 2 etapas
- En la primera etapa se copian los primeros 5 elementos mientras que en la segunda los 5 restantes.
- Cada etapa corresponde a un major loop del DMA
- En la interrupción del major loop se ingresan 5 elementos nuevos al buffer origen (para simular que hubo una nueva adquisición de datos).

Simulación


Condiciones iniciales
SRC
DST
0 1 2 3 4 5 6 7 8 9
0 0 0 0 0 0 0 0 0 0
SADD= SRC
DADD= DST
CITER=5
TCD A
Major Loop 1
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 0 0 0 0 0
SADD= SRC
DADD= DST
CITER=5
TCD A
New data
Load TCD B
Major Loop 2
A B C D E 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
SADD= SRC+5
DADD= DST+5
CITER=5
TCD B
Load TCD A
New data
Major Loop 3
A B C D E F 10 11 12 13
A B C D E 5 6 7 8 9
SADD= SRC
DADD= DST
CITER=5
TCD A
New data
Load TCD B
Major Loop 4
14 15 16 17 18 F 10 11 12 13
A B C D E F 10 11 12 13
SADD= SRC+5
DADD= DST+5
CITER=5
TCD B

Channel to channel Linking

The following table summarizes how a DMA channel can link to another DMA channel, i.e, use another channel's TCD, at the end of a loop.

Table 22-10. Channel Linking Parameters

Desired Link Behavior	TCD Control Field Name	Description
Link at end of Minor Loop	CITER[E_LINK]	Enable channel-to-channel linking on minor loop completion (current iteration)
	CITER[LINKCH]	Link channel number when linking at end of minor loop (current iteration)
Link at end of Major Loop	CSR[MAJOR_E_LINK]	Enable channel-to-channel linking on major loop completion
	CSR[MAJOR_LINKCH]	Link channel number when linking at end of major loop

Note

The TCD*n*_CITER[E_LINK] bit and the TCD*n*_BITER[E_LINK] bit must equal or a configuration error is reported. The CITER and BITER vector widths must be equal to calculate the major loop, half-way done interrupt point.